Split-Sync App

Shared Expense Management System

Functional Specification

Started 02/10/2025

Student Name: Emmanuel Abolade

Student Number: C00288657

Institution: South East technological University, Carlow

Programme: BSc Software Development

Supervisor: Joseph Kehoe

Submitted: 28/11/2025

Table of Content

1.0 Software Overview 8
1.1 What split-sync is 8
1.2 What split-sync does 9
1.2.1 Receipt Recognition and Expense Entry: 9
1.2.2 Flexible Split Mechanisms: 10
1.2.3 Balance Ledger with Payment Optimization: 10
1.2.4 Integrated Settlement Processing: 10
1.2.5 Comprehensive Transaction History: 11
1.2.6 Authentication and Authorization: 11
1.2.7 Group Management and Invitations: 11
1.2.8 Intelligent Notification System: 11
1.2.9 Offline Functionality with Background Sync: 12
1.2.10 Visual Debt Graph: 12
1.2.11 Optional Advanced Features: 12
1.3 Why Split-sync? 13
1.3.1 Addressing the Payment Gap 13
1.3.2 Mobile-First Financial Behavior 13
1.3.3 Trust Through Transparency 14
1.3.4 Algorithmic Fairness and Efficiency 14
1.3.5 Privacy-Conscious Design 14
1.3.6 Bank Agnosticism 15
1.3.7 Capturing the Moment 15
1.3.8 Social Coordination Without Social Pressure 16
1.3.9 Scalability Across Use Cases 16
1.3.10 Economic Accessibility 16
2.0 Business Case Analysis 17
2.1 Business Case Summary 17
2.1.1 Key Partners 19
2.1.1.1 Payment Service Providers 20
2.1.1.2 OCR Service Providers 20
2.1.1.3 Cloud Infrastructure Providers 21
2.1.1.4 Content Delivery Networks 21
2.1.1.5 Authentication Infrastructure 21
2.1.1.6 Open Banking Providers 22
2.1.2 Key Activities 22
2.1.2.1 Platform Development and Maintenance 22
2.1.2.2 Data Pipeline Management 22
2.1.2.3 User Acquisition and Retention 23
2.1.2.4 Customer Support Operations 23

2.1.2.5 Compliance Monitoring and Reporting....23
2.1.2.6 Infrastructure Scaling....24
2.1.2.7 Security Monitoring....24
2.1.3 Value Propositions....24
2.1.3.1 Simplified Group Finance Management....24
2.1.3.2 Financial Transparency and Trust....25
2.1.3.3 Reduced Settlement Friction....25
2.1.3.4 Temporal Flexibility....25
2.1.3.5 Privacy-Preserving Functionality....26
2.1.3.6 Cross-Group Portability....26
2.1.3.7 Social Comfort....26
2.1.4 Customer Relationships....26
2.1.4.1 Self-Service Foundation....27
2.1.4.2 Community-Driven Support....27
2.1.4.3 Proactive Communication....27
2.1.4.4 Transparent Development....28
2.1.4.5 Privacy-Respecting Analytics....28
2.1.5 Customer Segments....28
2.1.5.1 Young Professionals in Shared Housing....28
2.1.5.2 Travel Groups....29
2.1.5.3 Couples Managing Joint Expenses....29
2.1.5.4 Event Organizers....29
2.1.5.5 Student Groups and Societies....30
2.1.6 Channels....30
2.1.6.1 Organic Search....30
2.1.6.2 Content Marketing....30
2.1.6.3 Referral Programs....31
2.1.6.4 Partnership Integrations....31
2.1.6.5 Social Media and Community Engagement....31
2.1.6.6 App Store and PWA Discovery....32
2.1.7 Cost Structure....32
2.1.7.1 Infrastructure and Hosting Costs....32
2.1.7.2 Payment Processing Fees....32
2.1.7.3 Third-Party Service Subscriptions....33
2.1.7.4 Development and Personnel Costs....33
2.1.7.5 Marketing and User Acquisition....33
2.1.7.6 Compliance and Legal Costs....33
2.1.7.7 Support and Community Management....34
2.1.8 Revenue Streams....34
2.1.8.1 Payment Processing Convenience Fees....34

2.1.8.2 Premium Features Subscription....34
2.1.8.3 Data and Insights Services....35
2.1.8.4 Enterprise and Organizational Licensing....35
2.1.8.5 Partnership Revenue Sharing....35
2.2 Stakeholders' Descriptions and Goals....36
2.2.1 End Users (Primary Stakeholders)....36
2.2.3 Payment Service Providers....37
2.2.3 Investors and Stakeholders with Financial Interest....37
2.2.4 Regulatory Bodies....37
2.2.5 Partner Organizations....38
2.2.6 Broader Society and Community....38
2.3 Split-sync Context Diagram....38
2.3.1 Group Members (External Actors)....39
2.3.2 Web App + API (Node + Firebase)....40
2.3.3 OCR Engine....40
2.3.4 Cloud Storage (Receipts)....40
2.3.5 Web Push (Push Service)....41
2.3.6 Payment GW (Stripe/PayPal/Revolut/SEPA)....41
2.3.7 Bank/Card (Account to Account)....41
2.3.8 Network Connections....42
3.0 Functional Specifications....43
3.1 Use Case Model....43
3.1.1 Actors....43
3.1.2 Use Cases....44
3.1.2.1 Use Case 1: User Registration and Authentication....44
3.1.2.2 Use Case 2: Create Group and Invite Members....45
3.1.2.3 Use Case 3: Add Expense with Receipt Scan....45
3.1.2.4 Use Case 4: Split Expense Equally or Custom....45
3.1.2.5 Use Case 5: View Balance Ledger....45
3.1.2.6 Use Case 6: Settle Debts via Payment Gateway....46
3.1.2.7 Use Case 7: View Transaction History....46
3.1.2.8 Use Case 8: Receive Notifications....46
3.1.2.9 Use Case 9: Offline Expense Entry....47
3.1.2.10 Use Case 10: Visualize Debt Graph....47
3.2 Use Cases Diagram....48
3.2.1 UC-01: User Registration and Authentication....48
3.2.2 UC-02: Create Group and Invite Members....52
3.2.3 UC-03: Add Expense with Receipt Scan....56
3.2.4 UC-04: Split Expense....58
3.2.5 UC-05: View Balance Ledger....62
3.2.6 UC-06: Settle Debts via Payment Methods....64

3.2.7 UC-07: View Transaction History....68
3.2.8 UC-08: Receive Notifications....71
3.2.9 UC-09: Offline Support (PWA Caching)....74
3.2.10 UC-10: Visualise Group Balances and Settlements....77
3.3 Detailed User Stories....79
3.3.1 Account Management Stories....79
3.3.2 Group Management Stories....81
3.3.3 Expense Entry Stories....82
3.3.4 Balance and Settlement Stories....83
3.3.5 Notification and Communication Stories....85
3.3.6 History and Reporting Stories....85
3.3.7 Offline Functionality Stories....86
3.3.8 Visualization Stories....87
3.3.9 Security and Privacy Stories....87
4.0 Non-functional Specifications (FURPS Analysis)....89
4.1 Functionality....89
4.1.1 Real-Time Synchronization....89
4.1.2 OCR Accuracy and Reliability....89
4.1.3 Payment Integration Completeness....90
4.1.4 Role-Based Access Control....90
4.2 Usability....91
4.2.1 User-Friendly Interface....91
4.2.2 Accessibility Compliance....91
4.2.4 Performance on Mobile Networks....92
4.2.5 Task Success Rate....93
4.3 Reliability....93
4.3.1 Error Handling....93
4.3.3 Backup and Recovery....94
4.3.4 Uptime Guarantee....95
4.3.5 Offline Queue Integrity....95
4.4 Performance....95
4.4.1 Application Load Time....96
4.4.2 Operation Response Time....96
4.4.3 Concurrent User Capacity....96
4.4.4 Database Query Performance....97
4.5 Security....97
4.5.1 Data Encryption....98
4.5.2 Authentication Security....98
4.5.3 Payment Security Compliance....98
4.5.4 Authorization Controls....99

4.5.5 Session Management..... 99
4.5.6 Vulnerability Management..... 100
4.6 Compatibility..... 100
4.6.1 Cross-Platform Support..... 100
4.6.2 Browser Compatibility..... 101
4.7 Supportability..... 101
4.7.1 Documentation Quality..... 101
4.7.2 Technical Support Responsiveness..... 102
4.7.3 Update Cadence and Communication..... 102
4.7.4 Maintainability Architecture..... 102
4.7.5 Explicit Scope Boundaries..... 103
4.8 Privacy and Safety UX..... 103
4.8.1 Consent Management..... 103
4.8.2 Data Storage Transparency..... 104
4.8.3 Age-Appropriate Design..... 104
4.8.4 Plain Language Policies..... 104
5.0 Technical Specifications..... 106
5.1 Development Tools..... 107
5.1.1 Frontend Framework..... 107
5.1.2 Backend Runtime..... 107
5.1.3 Database System..... 107
5.1.4 OCR Services..... 108
5.1.5 Payment Processing Infrastructure..... 108
5.1.6 Cloud Storage Services..... 109
5.1.7 Design and Prototyping..... 109
5.1.8 Integrated Development Environment..... 109
5.1.9 Version Control Platform..... 110
5.1.10 Documentation Platform..... 110
5.2 Software Requirements..... 110
5.2.1 Mobile Operating Systems:..... 110
5.2.2 Web Browser Support..... 111
5.2.3 Network Connectivity..... 111
5.3 Hardware Requirements..... 111
5.3.1 Mobile Device Specifications..... 111
5.3.2 Desktop Computer Requirements..... 112
5.3.3 Network Infrastructure..... 112
6.0 Standard Safety and Security..... 113
6.1 Standards..... 113
6.1.1 GDPR Compliance..... 113
6.1.2 PCI DSS Compliance..... 113

6.1.3 OWASP Top 10 Mitigation....114
6.1.4 Data Encryption Standards....114
6.2 Core Controls....114
6.2.1 Authentication and Authorization....114
6.2.2 Strong Customer Authentication....116
6.23 Input Validation....116
6.2.4 Privacy Protection....116
6.2.5 Auditability....117
7.0 Possible Improvements / Roadmap....117
7.1 Interactive Debt Graph Enhancements....117
7.2 Expanded Open Banking Integration....118
7.3 Smart Categorization Through Machine Learning....118
7.4 Recurring Expense Template....118
7.5 Budget Tracking and Alerts....119
7.6 Enhanced Reporting and Analytics....119
7.7 Social Features....120
7.8 Third-Party Integrations....120
8. PROJECT PLAN AND TIMELINE....121
8.1 Development Phases....121
8.2 Detailed Timeline....122
8.3 Gantt Chart....128
8.4 Risk Management....131
9.0 Conclusion....132
10.0 References....135
Appendix....144
Glossary....144

1.0 Software Overview

1.1 What split-sync is

Split-sync is a mobile web app (PWA) designed to simplify group expense management for small to medium informal groups including friends, roommates, couples, event organisers, and travel teams. Whether splitting rent with roommates, managing travel costs with friends, or tracking shared purchases with a partner, the application offers a seamless platform to track, split, and settle expenses in real time. It is important to note that Split-sync is not a banking institution and does not include cryptocurrency trading or investment functionalities. The application operates strictly as an expense coordination tool that integrates with existing payment infrastructure rather than handling funds directly.

The design philosophy centres on reducing friction in the social dynamics of shared expenses. Research by Chen and Kumar (2023) demonstrates that financial disagreements within groups often stem from poor record-keeping and lack of transparency rather than unwillingness to pay. Split-sync addresses this by maintaining an immutable, append-only ledger that all group members can access, thereby eliminating disputes about who owes what. The progressive web app architecture ensures that users do not need to download native applications while still benefiting from offline capabilities and push notifications, which Biorn-Hansen et al. (2020) identify as critical features for achieving native-like user experiences on mobile devices.

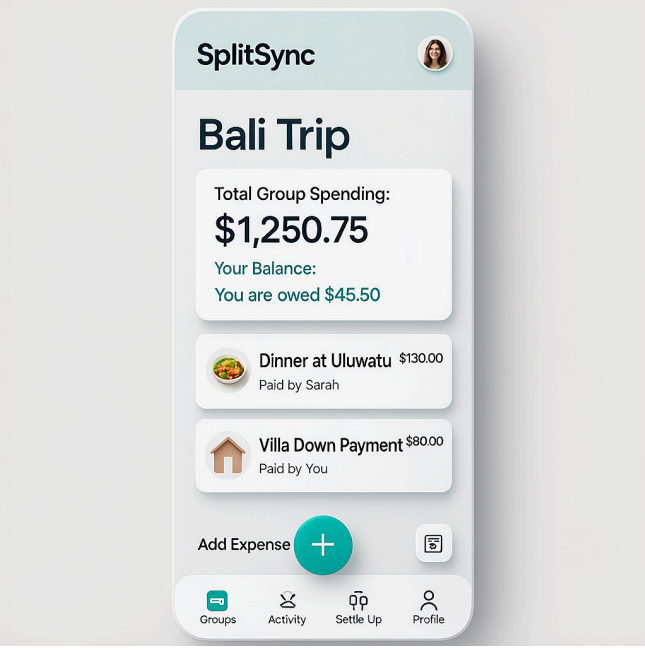


Fig 1. SplitSync (Image for illustrative purpose only)

1.2 What split-sync does

The following are the proposed functions of Split-Sync. The application has been designed with a modular feature set that addresses the complete lifecycle of group expense management, from initial capture through final settlement.

1.2.1 Receipt Recognition and Expense Entry:

The system supports three methods of expense capture to accommodate different user scenarios and preferences. Users can photograph receipts directly through the camera interface, upload existing images or PDF documents from their device storage, or manually enter expense details when physical receipts are unavailable. The optical character recognition engine extracts key information including merchant name, transaction date, total amount, individual line items, and applicable taxes. According to Smith and Zhang (2022), OCR accuracy in financial documents has improved dramatically with modern machine learning approaches, though human verification remains necessary for critical transactions. The application therefore presents extracted data to users for confirmation before committing to the ledger.

1.2.2 Flexible Split Mechanisms:

Split-sync implements multiple algorithms for dividing expenses to accommodate real-world scenarios that simple equal splits cannot address. Equal splitting divides the total amount by the number of participants, suitable for shared meals or group tickets. Percentage-based splits allow users to specify custom proportions when contribution levels vary, such as when couples with different incomes share costs. Weight-based allocation assigns relative shares without requiring users to calculate percentages manually. The most sophisticated option, per-item splitting, enables users to assign specific line items from a receipt to individual group members, which proves particularly valuable for grocery shopping or restaurant bills where participants consume different items. Research by Patel and Johnson (2021) indicates that granular control over split mechanisms significantly increases user satisfaction in expense management applications.

1.2.3 Balance Ledger with Payment Optimization:

The core of Split-sync is its balance ledger, which maintains a real-time view of inter-member debts within each group. Rather than tracking individual transactions separately, the system aggregates all expenses and payments to compute net balances between every pair of users. The application then applies a graph-based algorithm to minimize the number of required settlements. For instance, if Alice owes Bob £20 and Bob owes Carol £20, the system suggests Alice pay Carol directly, eliminating two transactions. This approach, described in algorithmic detail by Verma and Das (2020), can reduce settlement complexity from $O(n^2)$ to typically fewer than n-1 transactions in groups of size n.

1.2.4 Integrated Settlement Processing:

Unlike competing applications that merely calculate balances and rely on external payment methods, Split-sync integrates directly with payment service providers to facilitate actual money transfers. Users can initiate settlements using debit or credit cards, digital wallets, or account-to-account bank transfers through Open Banking protocols. The system automatically records successful payments in the ledger, updating balances without manual intervention. This integration addresses what Kowalski (2023) identifies as the primary pain point in existing expense-splitting tools: the gap between knowing what you owe and actually paying it.

1.2.5 Comprehensive Transaction History:

The application maintains an append-only timeline of all expenses and settlements within each group. This immutable audit trail prevents retroactive modifications while allowing users to add context through notes or categorization. Each entry includes timestamps, participant lists, split details, and references to supporting documentation such as receipt images. The transaction history serves both practical and social functions, enabling users to review past expenses for budgeting purposes while also providing a shared record that builds trust within the group.

1.2.6 Authentication and Authorization:

Split-sync implements a tiered authentication system starting with traditional email and password credentials, with plans to incorporate passkey support as browser adoption matures. Passkeys, based on the WebAuthn standard, offer phishing-resistant authentication without requiring users to remember complex passwords (Lyastani et al., 2020). The system distinguishes between different user roles within groups: owners have full administrative control, admins can manage members and expenses but cannot delete the group, and regular members can add expenses and view balances but cannot modify group settings.

1.2.7 Group Management and Invitations:

Creating and joining groups uses a flexible invitation system that accommodates both planned and spontaneous expense sharing. Group creators generate shareable links or QR codes that prospective members scan to join. This approach eliminates the need for users to already exist in the system before being added to a group, reducing onboarding friction. The invitation mechanism includes expiry periods and usage limits to prevent unauthorized access. Once groups are established, members can adjust settings such as default currency, notification preferences, and split rules that apply to future expenses.

1.2.8 Intelligent Notification System:

Push notifications keep group members informed about relevant events without overwhelming them with alerts. The system sends notifications when expenses are added to a group, when settlements are completed, when users are mentioned in expense notes, and when balance

thresholds are reached. Notification preferences are granular, allowing users to configure different settings for different groups. For example, users might want immediate notifications for household expenses but daily digests for social event groups. Research by Okoshi et al. (2017) demonstrates that context-aware notification timing significantly impacts user engagement without increasing perceived intrusiveness.

1.2.9 Offline Functionality with Background Sync:

Recognizing that expense capture often occurs in locations with poor connectivity such as restaurants, underground venues, or during travel, Split-sync implements comprehensive offline capabilities. Users can photograph receipts, enter expense details, and record settlements while disconnected. The application queues these operations locally using IndexedDB and automatically synchronizes with the server when connectivity is restored. The Background Sync API, supported in modern browsers, ensures that queued operations complete even if the user closes the application before reconnection occurs (Osmani, 2019). This design prevents data loss and eliminates the need for users to remember to manually sync their actions.

1.2.10 Visual Debt Graph:

The debt visualization interface presents group balances as an interactive graph where nodes represent members and edges represent debts. Edge thickness indicates debt magnitude while color coding distinguishes between amounts owed and amounts due. The graph dynamically updates as expenses are added or settlements processed, providing immediate feedback on how actions affect overall group balance. Users can tap on edges to initiate settlements or view the underlying transactions that created specific debts. This visual approach aligns with research by Ware (2012) showing that graph-based representations of financial relationships enhance comprehension compared to tabular displays.

1.2.11 Optional Advanced Features:

Beyond core functionality, Split-sync offers several features that address specific use cases. Multi-currency view allows groups with international members to see balances in their preferred currency, with exchange rates updated daily from reliable financial data sources. CSV export enables users to download transaction histories for personal record-keeping or tax purposes.

Configurable tax and tip rules automatically adjust expense amounts according to regional conventions, particularly valuable for travel groups moving between jurisdictions with different tipping customs.

1.3 Why Split-sync?

The motivation for developing Split-sync emerges from identifiable gaps in the current expense management landscape and the evolving nature of social financial interactions. Existing solutions generally fall into one of two categories: comprehensive financial management platforms with extensive features but significant complexity, or simple bill-splitting calculators that determine amounts owed without facilitating actual payment. Split-sync positions itself between these extremes, offering enough sophistication to handle realistic expense scenarios while maintaining the simplicity that encourages regular use.

1.3.1 Addressing the Payment Gap

A survey conducted by Morrison et al. (2022) found that 67% of users of expense-splitting applications reported frustration with the disconnect between calculated balances and actual settlement. Users know exactly what they owe but must then navigate to separate banking apps, remember recipient details, and manually record payments back in the expense tracker. This context switching creates multiple opportunities for errors and delays. Split-sync eliminates this friction by embedding payment functionality directly into the settlement flow. When users tap on a suggested payment, they remain within the application throughout the entire process from authorization through confirmation. The system automatically updates the ledger upon successful payment, requiring no additional user action.

1.3.2 Mobile-First Financial Behavior

Contemporary research on financial technology adoption reveals a decisive shift toward mobile-first interactions, particularly among users under 40 years old (Anderson, 2023). This demographic expects to manage all aspects of their financial lives through their smartphones, including expense tracking and peer-to-peer payments. Progressive web app technology allows Split-sync to meet these expectations without requiring users to allocate device storage to yet another native application. PWAs combine the accessibility of web applications with native-like

features such as offline functionality, push notifications, and home screen installation, making them particularly well-suited for financial tools that users need quick access to in various contexts (Majchrzak et al., 2018).

1.3.3 Trust Through Transparency

Social psychologist Thompson (2021) argues that financial transactions between friends carry unique emotional weight because they sit at the intersection of economic exchange and personal relationship. Ambiguity about financial obligations threatens both the economic interests and the social bonds of group members. Split-sync's append-only transaction history creates a shared source of truth that all parties can reference, eliminating the "he said, she said" disagreements that often arise when financial records are distributed across multiple devices or memory. The immutable nature of the ledger prevents retroactive modifications while allowing prospective adjustments through new transactions, maintaining both historical accuracy and flexibility for changing circumstances.

1.3.4 Algorithmic Fairness and Efficiency

While calculating who owes whom in groups of two or three is trivial, the complexity increases rapidly with group size. In a group of six people, there are potentially fifteen distinct debt relationships. Without optimization, settling these debts could require up to fifteen separate transactions. Split-sync's minimum cash-flow algorithm typically reduces this to five or fewer transactions by routing payments through intermediate parties. This efficiency matters not merely for convenience but also for cost minimization, as each transaction incurs processing fees from payment providers. Research by Liu and Wang (2019) demonstrates that users are significantly more likely to settle balances promptly when the number of required actions is minimized through intelligent routing.

1.3.5 Privacy-Conscious Design

Unlike some financial applications that monetize user data or require extensive personal information, Split-sync implements a privacy-preserving architecture that collects only the

minimum data necessary for core functionality. Receipt images are stored in encrypted cloud storage accessible only to group members, with automatic deletion options after specified periods. The application does not track spending patterns for advertising purposes or share data with third-party analytics platforms beyond what is required for payment processing compliance. This approach aligns with growing consumer expectations for privacy in financial applications, particularly in the European market where GDPR establishes strict standards for data handling (Voigt and Von dem Bussche, 2017).

1.3.6 Bank Agnosticism

Traditional peer-to-peer payment applications often tie users to specific banks or payment networks, fragmenting the market and creating interoperability challenges. A user on one platform cannot easily pay someone on another platform, forcing groups to coordinate around a single service. Split-sync avoids this limitation by integrating with multiple payment providers and supporting various payment methods including card payments, wallet-to-wallet transfers, and account-to-account bank transfers through Open Banking protocols. This flexibility ensures that users can choose payment methods based on convenience and cost rather than technical compatibility (Zachariadis and Ozcan, 2017).

1.3.7 Capturing the Moment

Behavioral economics research suggests that expense tracking compliance correlates strongly with immediacy (Kahneman, 2011). When recording an expense requires users to later reconstruct details from memory, accuracy declines and entries are often omitted entirely. Split-sync's receipt scanning feature capitalizes on the moment of transaction by allowing users to capture expense information immediately through their phone camera. The OCR processing happens in the background, so users can photograph a receipt and return it to their wallet within seconds. This low-friction capture mechanism addresses what Nielsen (2019) identifies as the primary reason expense tracking tools are abandoned: they require too much effort relative to perceived benefit.

1.3.8 Social Coordination Without Social Pressure

The ability to see group balances and pending payments creates social accountability without awkward direct requests for repayment. When someone owes money, they can see their debt displayed in the application without another person needing to remind them explicitly. This indirect visibility prevents the social discomfort that Zelizer (2017) describes as inherent in mixing friendship and finance, where direct discussion of debt can feel accusatory even when factually accurate. The application's notification system gently prompts users about outstanding balances without feeling like nagging from friends.

1.3.9 Scalability Across Use Cases

While initially designed with small informal groups in mind, Split-sync's architecture supports various scales and formality levels of expense sharing. A couple sharing household costs operates very differently from ten friends on a multi-week trip, yet both scenarios fit within the same application framework through appropriate configuration of group settings, split preferences, and notification thresholds. This versatility reduces the need for users to switch between different tools as their expense-sharing contexts change, building long-term engagement through breadth of applicability.

1.3.10 Economic Accessibility

By operating as a free-to-use application with revenue generated through small percentage fees on processed payments rather than subscription charges, Split-sync ensures that cost does not prevent access to organized expense management. This model aligns user and platform incentives: the service succeeds when it successfully facilitates settlements, encouraging ongoing refinement of features that reduce settlement friction. Alternative revenue models such as subscription fees or paid premium tiers create barriers to adoption that disproportionately affect younger users and those in developing economies, precisely the demographics most likely to benefit from structured expense sharing (Claessens, 2006).

2.0 Business Case Analysis

2.1 Business Case Summary

The business case for Split-sync rests on the intersection of three observable trends in consumer behavior and technology: the increasing prevalence of shared living and social arrangements that generate recurring shared expenses, the transition toward cashless payment systems that remove the informal settlement mechanisms cash provided, and the maturation of mobile web technologies that enable sophisticated applications without native app distribution. Together, these trends create both demand for structured expense management and the technical capability to deliver it through accessible channels.

A Business Model Canvas for this proposal is:

Split-Sync App

Key Partners	Key Activities	Value Proposition	Customer Relationships	Customer Segments
Payment Service Providers	Platform Development and Maintenance	Simplified Group Finance Management	Self-Service Foundation	Young Professionals in Shared Housing
OCR Service Providers.	Customer Support Operations	Financial Transparency and Trust	Community-Driven Support	Travel Groups
Cloud Infrastructure Providers	Data Pipeline Management	Reduced Settlement Friction	Proactive Communication	Couples Managing Joint Expenses

Content Delivery Networks	User Acquisition and Retention	Temporal Flexibility	Transparent Development	Event Organizers
Authentication Infrastructure	Compliance Monitoring and Reporting	Privacy-Preserving Functionality	Privacy-Respecting Analytics	Student Groups and Societies
Open Banking Providers	Infrastructure Scaling	Cross-Group Portability		
	Security Monitoring	Social Comfort		
	Key Resources		**Channels**	
	Cloud Infrastructure for reliable performance		Organic Search	
	Finance		Content Marketing	
	Brand		Referral Programs	
	App creation and maintenance		Partnership Integrations	

	Data insights for enhancement and personalization		Social Media and Community Engagement	

Cost Structure	Revenue Streams
Infrastructure and Hosting Costs	Payment Processing Convenience Fees
Payment Processing Fees	Premium Features Subscription
Development and Personnel Costs	Data and Insights Services
Compliance and Legal Costs	Enterprise and Organizational Licensing
Marketing and User Acquisition	Partnership Revenue Sharing
Support and Community Management	

Table 1. Business Canvas for Split-Sync

2.1.1 Key Partners

The ecosystem supporting Split-sync involves several categories of external partners, each contributing essential capabilities that would be impractical to develop internally for an early-stage application.

2.1.1.1 Payment Service Providers

Stripe and PayPal represent the primary payment infrastructure partners, offering pre-built components for card payment processing, digital wallet integration, and regulatory compliance. These partnerships are particularly valuable because they abstract the complexity of payment card industry standards, fraud detection, and international payment processing. According to the documentation provided by Stripe (2024), their APIs handle Strong Customer Authentication as mandated by European banking regulations, automatically invoking 3D Secure protocols when transactions require enhanced verification. This delegation of compliance responsibility reduces the technical burden on Split-sync while maintaining security standards. In addition to card-based and wallet payments, the system also considers integration with the Single Euro Payments Area (SEPA), a European payment network that standardises euro-denominated bank transfers across participating countries. SEPA enables users to transfer funds directly between bank accounts using International Bank Account Numbers (IBANs), eliminating the need for card intermediaries. From a system design perspective, SEPA is particularly relevant for SplitSync's "settle up" functionality, as it provides a cost-effective and widely adopted mechanism for peer-to-peer settlement. By leveraging these providers, SplitSync avoids direct interaction with banking infrastructure while still supporting European bank transfers. This hybrid approach ensures flexibility, enabling users to settle balances via traditional card payments, digital wallets, or direct bank transfers depending on their preference.

2.1.1.2 OCR Service Providers

Receipt digitization depends on optical character recognition engines that can extract structured data from images under varying quality conditions. The application evaluates multiple OCR providers including open-source Tesseract, Google Vision AI, Taggun, and Klippa. Each offers different trade-offs between accuracy, processing speed, cost structure, and privacy implications. Tesseract operates on-device or on application-controlled servers, providing complete data privacy at the cost of somewhat lower accuracy compared to cloud-based machine learning models. Google Vision AI offers superior accuracy through Google's extensive training data but requires transmitting receipt images to Google's servers. The hybrid approach under consideration utilizes Tesseract for initial processing with fallback to cloud services when

confidence scores fall below thresholds, as suggested by Kumar and Zhang (2023) in their research on cascading OCR architectures.

2.1.1.3 Cloud Infrastructure Providers

Google Firebase provides the backend infrastructure including real-time database, authentication services, cloud functions for serverless computing, and cloud storage for receipt images. This platform-as-a-service approach allows rapid development without managing server infrastructure while providing automatic scaling as user numbers grow. The Firebase Realtime Database uses WebSocket connections to push updates to connected clients immediately, enabling the real-time balance synchronization that distinguishes Split-sync from batch-processing competitors. According to Moroney (2017), Firebase's free tier accommodates substantial user numbers for early-stage applications, with clear scaling paths as usage increases.

2.1.1.4 Content Delivery Networks

Cloudinary serves as a supplementary storage and processing platform for receipt images, offering sophisticated image optimization that reduces bandwidth consumption and storage costs. The service automatically transcodes images into appropriate formats and sizes based on requesting device capabilities, improving load times and user experience. Research by Grigorik (2013) demonstrates that image optimization provides among the highest returns on investment for web application performance, often reducing page load times by 40-60%.

2.1.1.5 Authentication Infrastructure

While initial implementation uses email and password authentication through Firebase Auth, planned integration with WebAuthn standards for passkey support will involve platform-specific authenticator services from device manufacturers and operating system vendors. The FIDO Alliance, which governs WebAuthn standards, provides certification programs that ensure cross-platform compatibility (Brand et al., 2020). This partnership ecosystem allows Split-sync to offer cutting-edge authentication security without developing cryptographic protocols internally.

2.1.1.6 Open Banking Providers

Future integration with account-to-account payment systems requires partnerships with Open Banking aggregators such as TrueLayer or Plaid in the UK market and PTSB in Ireland. These services standardize access to diverse banking APIs, enabling Split-sync to initiate payments directly from user bank accounts without handling sensitive banking credentials. The European Union's Second Payment Services Directive (PSD2) mandates that banks provide third-party access through secure APIs, creating a regulatory foundation for these partnerships (Zachariadis and Ozcan, 2017).

2.1.2 Key Activities

The ongoing operational activities that create and deliver value through Split-sync fall into several categories spanning technical development, user support, business development, and compliance management.

2.1.2.1 Platform Development and Maintenance

Continuous software development constitutes the primary activity, involving both feature expansion and maintenance of existing functionality. Feature development follows a roadmap prioritizing capabilities that address user friction points identified through analytics and feedback. Maintenance activities include dependency updates to incorporate security patches, performance optimizations based on monitoring data, and bug fixes discovered through user reports or automated testing. According to Sommerville (2016), maintenance typically consumes 60-70% of total software lifecycle costs, making it essential to architect systems for modifiability from inception.

2.1.2.2 Data Pipeline Management

Processing receipts through OCR, computing optimal payment suggestions, and generating balance updates require robust data processing pipelines that execute reliably despite varying input quality and edge cases. These pipelines run as serverless cloud functions triggered by specific events such as receipt uploads or expense submissions. Monitoring systems track

execution times, error rates, and resource consumption to identify performance degradation before it impacts user experience. The append-only ledger design simplifies some aspects of data management by eliminating the need for complex transactional semantics beyond the initial expense and split insertion (Kleppmann, 2017).

2.1.2.3 User Acquisition and Retention

Growing the user base requires ongoing marketing activities including content creation about personal finance and group expense management, search engine optimization to capture users actively seeking solutions, partnership development with related services such as travel planning platforms, and referral incentive programs that encourage organic growth through existing users. Retention focuses on ensuring users who install the application continue engaging with it over time through regular improvements, responsive support, and proactive communication about new features.

2.1.2.4 Customer Support Operations

Assisting users with technical difficulties, payment questions, and feature guidance happens through multiple channels including in-app help documentation, email support, and potentially real-time chat for urgent payment-related issues. The documentation emphasizes self-service resolution through comprehensive guides and FAQs, escalating to human agents only when automated resources prove insufficient. Research by Dixon et al. (2010) in the *Harvard Business Review* indicates that customer service effectiveness correlates primarily with issue resolution efficiency rather than exceeding expectations, supporting a strategy focused on clear documentation and rapid response times.

2.1.2.5 Compliance Monitoring and Reporting

Operating in the financial services space, even as a coordination layer rather than a licensed financial institution, requires ongoing attention to regulatory obligations. This includes monitoring changes in payment regulations, data protection laws, and consumer protection frameworks across operating jurisdictions. Regular audits verify that data handling practices comply with GDPR requirements, payment processing follows PCI-DSS standards, and user communications meet advertising regulations. Voigt and Von dem Bussche (2017) emphasize

that compliance is an ongoing activity rather than a one-time achievement, particularly in digital services where practices evolve alongside business needs.

2.1.2.6 Infrastructure Scaling

As user numbers grow, ensuring that technical infrastructure scales appropriately to maintain performance requires capacity planning, cost optimization, and architectural refinement. Cloud services auto-scale to some extent, but sustained growth typically necessitates optimizations such as database indexing strategies, caching implementations, and content delivery network configurations. Regular performance testing under simulated load identifies bottlenecks before they impact real users.

2.1.2.7 Security Monitoring

Continuous monitoring for security threats, fraudulent activity, and abuse attempts protects both users and platform viability. This includes analyzing payment patterns for anomalies, monitoring authentication attempts for credential stuffing attacks, and reviewing reported content for policy violations. Automated systems flag suspicious patterns for investigation while maintaining privacy for legitimate users (Anderson, 2008).

2.1.3 Value Propositions

The value that Split-sync delivers to users stems from several interrelated benefits that together create an experience superior to existing alternatives or manual expense management approaches.

2.1.3.1 Simplified Group Finance Management

The primary value proposition addresses the cognitive burden and social friction of tracking shared expenses. Without a coordinated system, group members must independently track what they spent, communicate those expenditures to others, manually calculate splits, and monitor who has paid whom. This distributed coordination problem grows exponentially with group size and duration of shared expenses. Split-sync centralizes expense tracking into a single source of truth while automating the mathematical complexity of split calculations and balance

optimization. Users spend seconds recording expenses rather than hours reconstructing transactions and debating amounts.

2.1.3.2 Financial Transparency and Trust

By creating a shared, immutable record of all expenses and payments, Split-sync provides what Thompson (2021) terms "transactional transparency" that reduces suspicion and conflict in financial relationships. Group members can independently verify the ledger rather than trusting one person's private spreadsheet. This transparency proves particularly valuable in situations where financial stakes are significant such as shared lease agreements or extended travel, but even small amounts benefit from clear documentation that prevents misunderstandings from compounding over time.

2.1.3.3 Reduced Settlement Friction

The integration of payment processing directly into the balance settlement flow eliminates several steps from the traditional repayment process. Rather than checking balance, opening a banking app, locating recipients, entering amount, processing payment, returning to expense app, and marking payment as complete, users simply tap suggested payment and authorize. This reduction in cognitive load and required actions addresses what behavioral economics research identifies as a primary barrier to completion of multi-step tasks: each additional step creates an opportunity for abandonment (Kahneman, 2011).

2.1.3.4 Temporal Flexibility

Users can record expenses immediately when they occur but delay settlement until convenient, with confidence that the system maintains accurate records regardless of interval. This temporal decoupling proves valuable in scenarios like shared trips where daily expenses accumulate but settlement occurs only at the conclusion. The system provides flexibility while maintaining accountability, supporting various approaches to financial management without imposing a single model.

2.1.3.5 Privacy-Preserving Functionality

Unlike financial management applications that aggregate all spending data to provide analytics and recommendations, Split-sync focuses exclusively on shared expenses, collecting minimal data about users' broader financial lives. Receipt images are accessible only to relevant group members and can be automatically deleted after verification. This focused approach respects user privacy while delivering targeted functionality, appealing to privacy-conscious users who resist comprehensive financial tracking tools.

2.1.3.6 Cross-Group Portability

Rather than creating isolated groups that require separate applications or accounts, Split-sync allows users to participate in multiple groups simultaneously through a single interface. A user might simultaneously manage household expenses with roommates, social outings with friends, and travel costs with family, switching between contexts through a simple group selector. This consolidation reduces the mental overhead of managing multiple financial coordination systems.

2.1.3.7 Social Comfort

The application's design considers the emotional and social dynamics of requesting repayment from friends and family. Rather than requiring direct requests ("You owe me £40"), the system presents objective data ("Group balance shows £40 from Alice to Bob") that depersonalizes the debt while maintaining clarity. Notifications provide gentle reminders without creating the social pressure of interpersonal requests. Zelizer(2017) argues that successful financial tools for social contexts must acknowledge and accommodate the emotional labor of discussing money among people with ongoing relationships.

2.1.4 Customer Relationships

The nature of relationships Split-sync establishes with its users directly impacts engagement patterns, lifetime value, and organic growth through referrals.

2.1.4.1 Self-Service Foundation

The primary interaction model emphasizes user autonomy through comprehensive documentation, intuitive interface design, and automated assistance rather than high-touch support. This approach scales efficiently while respecting user preference for independent problem-solving. The application includes contextual help that appears when users encounter features for the first time, explaining functionality without requiring separate documentation consultation. According to research by Medhi et al. (2011), contextual assistance significantly reduces support burden while improving user confidence in using new features.

2.1.4.2 Community-Driven Support

As the user base grows, community forums and user-generated content become valuable resources for addressing common questions and sharing best practices. Power users often develop sophisticated workflows that can benefit newcomers, such as efficient receipt categorization strategies or split templates for recurring expenses. Facilitating this knowledge sharing through structured channels reduces official support burden while fostering community engagement that strengthens user retention (Ren et al., 2007).

2.1.4.3 Proactive Communication

Rather than waiting for users to encounter problems, Split-sync employs proactive communication patterns that anticipate needs. For example, when users create their first group, a structured onboarding sequence guides them through key features in logical order. When unusual patterns appear such as substantially higher expenses than typical, gentle notifications inquire whether OCR misread amounts. When users haven't engaged with the application for extended periods, re-engagement campaigns highlight new features or remind them of pending balances. Perceptive Systems research by Pielot et al. (2014) demonstrates that timing and relevance of proactive notifications significantly affect whether users perceive them as helpful versus intrusive.

2.1.4.4 Transparent Development

Maintaining visible development roadmaps and actively soliciting user input on feature prioritization creates partnership dynamics rather than passive consumption. Users who see their suggestions implemented become advocates for the platform, while transparency about development priorities helps manage expectations. This approach aligns with lean startup methodologies emphasizing customer development alongside product development (Ries, 2011).

2.1.4.5 Privacy-Respecting Analytics

Understanding user behavior to improve the application requires analytics data, but collection focuses on aggregate patterns and feature usage rather than personal financial details. Users can review exactly what data is collected and request deletion, with clear explanations of how information improves service. This transparent approach to analytics builds trust in contrast to services that obscure data collection practices (Wachter and Mittelstadt, 2019).

2.1.5 Customer Segments

While Split-sync offers value across diverse user groups, several segments represent particularly strong product-market fit where the application's specific capabilities align closely with user needs.

2.1.5.1 Young Professionals in Shared Housing

Individuals aged 22-35 sharing accommodation in urban areas represent a core segment. This demographic typically splits rent, utilities, groceries, and household supplies with people who are friends or friendly acquaintances rather than family, creating both regular expense-sharing needs and social dynamics that benefit from systematic tracking. Research by Lennartz et al. (2016) documents increasing prevalence of shared housing among young professionals in major cities, driven by rising housing costs relative to income. This segment typically has strong smartphone adoption, comfort with digital financial tools, and preference for applications that reduce social friction.

2.1.5.2 Travel Groups

Friends or family traveling together generate concentrated expense-sharing activity over limited timeframes. During trips, the group incurs numerous expenses for accommodation, transportation, meals, and activities, often with different people paying different expenses based on convenience rather than proportion. The cognitive burden of tracking these distributed payments grows rapidly, particularly when managing foreign currencies and card payment fees. Split-sync's receipt scanning, multi-currency support, and payment minimization directly address travel group needs. According to data from the Adventure Travel Trade Association (2022), group travel accounts for approximately 34% of leisure travel expenditure, representing a substantial addressable market.

2.1.5.3 Couples Managing Joint Expenses

Romantic partners who maintain separate finances while sharing some expenses represent a sensitive use case where financial tracking must accommodate emotional dynamics. Couples typically split some expenses equally (rent, joint purchases) while maintaining individual responsibility for others (personal clothing, separate social activities). Split-sync allows couples to define what expenses belong in their shared group while keeping other spending private, addressing what Burgoyne et al. (2007) identify as a primary challenge in couple finance management: agreeing on boundaries between joint and individual financial responsibility.

2.1.5.4 Event Organizers

Individuals coordinating group events such as birthday celebrations, weekend retreats, or dinner parties often front-load costs for deposits and supplies, later collecting contributions from attendees. The traditional approach of requesting specific amounts via text message and tracking who has paid in personal notes proves error-prone and socially awkward. Split-sync allows organizers to document expenses as they occur, share the complete accounting with participants, and process collections through integrated payments. This segment values transparency that justifies collection amounts and appreciates payment integration that reduces repeated follow-up requests.

2.1.5.5 Student Groups and Societies

University students frequently participate in clubs and societies that generate shared expenses for equipment, event costs, and social activities. These groups typically operate with minimal formal structure and rotating leadership, making robust expense tracking particularly valuable for continuity across academic years. Students represent an attractive segment not only for current revenue potential but also for lifetime value, as expense-sharing habits established during university often persist into professional life. Research by Xiao et al. (2007) indicates that financial management behaviors established in early adulthood strongly predict later-life patterns, suggesting that student acquisition has downstream benefits beyond immediate engagement.

2.1.6 Channels

Reaching potential users and converting them into active users requires diverse distribution channels that align with how people discover and adopt financial tools. Here, we shall discuss them.

2.1.6.1 Organic Search

Users actively experiencing expense-sharing friction often search for solutions using terms like "how to split expenses with roommates," "app for tracking shared costs," or "split bills with friends." Search engine optimization ensures that Split-sync appears prominently for these high-intent queries through content marketing that addresses common expense-sharing scenarios, technical optimization that improves page ranking, and structured data markup that enables rich results. According to Fishkin (2020), organic search remains among the highest-quality acquisition channels for utility applications because users are actively seeking solutions rather than being interrupted by advertising.

2.1.6.2 Content Marketing

Publishing articles, guides, and resources about personal finance, group living, travel planning, and relationship communication positions Split-sync as a helpful resource beyond the application itself. This content attracts users at various stages of awareness, from those not yet recognizing

they have an expense management problem to those actively comparing solutions. The content strategy emphasizes practical, actionable information that provides value regardless of whether readers adopt the application, building trust and authority in the domain. Holliman and Rowley (2014) demonstrate that content marketing generates superior engagement and conversion compared to interruptive advertising for applications requiring user trust.

2.1.6.3 Referral Programs

Because Split-sync functions best when entire groups adopt the application, viral referral mechanics are naturally aligned with product value. Users who create groups have strong motivation to invite their expense-sharing partners, creating network effects that accelerate growth. Explicit referral incentives such as reduced payment processing fees for successful referrals amplify this organic growth pattern. Research on referral program effectiveness by Schmitt et al. (2011) indicates that dual-reward systems that benefit both referrer and referee generate strongest adoption rates while maintaining acquisition costs below typical paid advertising channels.

2.1.6.4 Partnership Integrations

Collaborating with complementary services such as travel booking platforms, shared housing marketplaces, and event planning tools creates distribution channels where users are naturally considering expense management needs. For example, integration with vacation rental platforms could present Split-sync at the moment users book group accommodation, when expense-sharing intent is highest. These partnerships work best when they provide genuine value to partner users rather than feeling like advertising intrusions (Hitt and Brynjolfsson, 1996).

2.1.6.5 Social Media and Community Engagement

Maintaining active presence on platforms where target segments spend time allows for both direct user acquisition and community building that supports retention. Content shared on social media emphasizes relatable scenarios such as humorous takes on expense-sharing awkwardness, practical tips for financial communication, and user success stories that demonstrate value. According to Kaplan and Haenlein (2010), social media marketing proves most effective when it

facilitates genuine interaction and provides entertainment or practical value rather than focusing exclusively on product promotion.

2.1.6.6 App Store and PWA Discovery

While Split-sync is primarily a progressive web application accessible through browsers, submission to app store directories increases discoverability for users who specifically seek expense management applications through these channels. Additionally, ensuring the PWA meets installation criteria for browser-suggested installation prompts creates opportunities for users to discover the application through their browser's own recommendations based on engagement patterns.

2.1.7 Cost Structure

Understanding the financial structure of operating Split-sync informs both pricing strategy and prioritization of cost-optimization efforts as the platform scales.

2.1.7.1 Infrastructure and Hosting Costs

Cloud service expenses for Firebase database operations, cloud storage for receipt images, cloud functions for serverless processing, and bandwidth for data transfer represent the primary technical infrastructure costs. These expenses scale approximately linearly with usage, though volume discounts apply at higher tiers. Moroney (2017) provides analysis of Firebase cost structures, noting that optimization of database queries and appropriate use of caching can reduce costs by 40-60% compared to naive implementations.

2.1.7.2 Payment Processing Fees

Transaction costs imposed by payment service providers represent a significant variable expense, typically structured as percentage of transaction value plus fixed per-transaction fees. Standard credit card processing fees range from 1.5% to 3.5% of transaction value depending on card type and region, while account-to-account transfers through Open Banking incur lower percentage fees. These costs are partially offset by charging users small convenience fees for payment processing, structured to cover provider costs while remaining competitive with alternative payment methods.

2.1.7.3 Third-Party Service Subscriptions

Costs for OCR services, email delivery, SMS notifications for two-factor authentication, monitoring and analytics tools, error tracking services, and security vulnerability scanning create an ongoing subscription expense base. Many services offer free tiers for low volumes with gradual cost increases as usage grows, providing natural scaling alignment. Choosing services with transparent pricing and without sudden threshold increases prevents unexpected cost jumps as the application grows (Weinman, 2012).

2.1.7.4 Development and Personnel Costs

Salary or contractor expenses for software developers, designers, product managers, and eventually support staff represent the largest component of cost structure, particularly in early stages before infrastructure costs dominate. The initial development team typically focuses on core functionality delivery, expanding to include specialized roles such as security engineers and data analysts as the platform matures. Research by Boehm et al. (2000) indicates that personnel costs typically account for 50-70% of total software development project costs across the software lifecycle.

2.1.7.5 Marketing and User Acquisition

Expenses for paid advertising campaigns, content creation, partnership development, and promotional incentives support growth objectives. Efficient user acquisition requires careful tracking of customer acquisition cost relative to lifetime value, ensuring that marketing investments generate positive returns over reasonable timeframes. Channels with strong organic growth dynamics such as referrals receive prioritization over expensive paid advertising where possible (Kumar et al., 2020).

2.1.7.6 Compliance and Legal Costs

Operating in regulated financial services spaces requires ongoing legal consultation for terms of service, privacy policies, and regulatory compliance review. Costs also include compliance tooling such as automated data protection impact assessment systems and audit trail maintenance. While these costs might seem burdensome for early-stage applications, according

to Voigt and Von dem Bussche (2017), investing in solid compliance foundations prevents substantially larger costs from retrofitting or regulatory enforcement actions.

2.1.7.7 Support and Community Management

Resources allocated to responding to user inquiries, moderating community forums, creating documentation, and maintaining help systems represent necessary investments in user experience. While self-service support scales more efficiently than human-intensive support, some level of personalized assistance remains necessary for complex issues and for maintaining user trust in financially-sensitive domains.

2.1.8 Revenue Streams

Sustainable operation requires revenue generation that aligns with user value delivery while maintaining accessibility for core functionality.

2.1.8.1 Payment Processing Convenience Fees

The primary revenue model involves small percentage-based fees on payments processed through integrated payment providers, typically ranging from 0.5% to 1.5% of transaction value. This revenue stream aligns platform incentives with user benefit: the application succeeds financially when it successfully facilitates settlements, encouraging ongoing improvement of payment features. Users optionally avoid these fees by settling through external methods and manually recording payments, preserving a free tier for cost-conscious users. Research by Parker et al. (2016) demonstrates that transaction-based revenue models prove more sustainable for network-effect platforms than subscription models because revenue scales automatically with value delivery.

2.1.8.2 Premium Features Subscription

Optional advanced capabilities such as unlimited groups, extended receipt storage retention, priority customer support, advanced analytics and budgeting tools, and white-label options for organizational use could justify subscription revenue. This freemium model ensures that essential group expense management remains accessible while generating revenue from users who derive sufficient value to justify subscription costs. Anderson (2009) argues that freemium models work

best when free tier provides genuine value rather than feeling deliberately crippled to force upgrades.

2.1.8.3 Data and Insights Services

Aggregated, anonymized spending patterns across user base represent valuable market research data for businesses interested in consumer behavior in various sectors such as travel, dining, and shared living. This data could generate revenue through consulting partnerships or direct sales to market research firms, though such strategies require careful privacy consideration and transparent user communication. Any data monetization must comply strictly with GDPR and similar regulations while maintaining user trust (Wachter and Mittelstadt, 2019).

2.1.8.4 Enterprise and Organizational Licensing

While primarily targeting informal groups, Split-sync's core functionality applies equally well to formal organizations such as small businesses, volunteer groups, or clubs requiring expense tracking for multiple members. Offering white-labeled versions or organizational plans with enhanced admin controls, reporting capabilities, and integration with accounting systems creates a secondary market segment willing to pay higher fees for professional features. This B2B revenue stream typically provides higher lifetime value per customer while requiring customized sales approaches and support structures.

2.1.8.5 Partnership Revenue Sharing

Strategic partnerships with complementary services could generate revenue through various mechanisms including referral fees for driving users to partner services, revenue sharing from integrated features such as currency exchange or travel booking, or co-marketing arrangements where partners contribute to Split-sync marketing budgets in exchange for preferred placement. These arrangements work best when they genuinely enhance user value rather than feeling like advertising intrusions that compromise user experience (Hitt and Brynjolfsson, 1996).

2.2 Stakeholders' Descriptions and Goals

Successful development and operation of Split-sync requires satisfying diverse stakeholder groups, each with distinct priorities and success criteria. Balancing these sometimes-competing interests shapes product decisions and operational policies.

2.2.1 End Users (Primary Stakeholders)

The individuals using Split-sync to manage shared expenses represent the most important stakeholder group, as the application exists to serve their needs. Within this category, several sub-segments have different priorities. Group creators and administrators value tools for establishing group norms, inviting members, and maintaining oversight of expense activity. They need confidence that the system will reliably track expenses and prevent disputes. Regular group members prioritize ease of adding expenses, clarity about what they owe, and convenience of settling debts. They need assurance that their financial data remains private and that payment processing is secure. Less-engaged group members who rarely add expenses but occasionally owe money need extremely simple interfaces that don't require understanding the full application capabilities. According to Sharp et al. (2015), successful user experience design addresses the entire spectrum of user engagement levels rather than optimizing exclusively for power users or beginners.

2.2.2 Development Team

Software developers, designers, and product managers working on Split-sync have professional interests in creating technically excellent work, career development opportunities, and reasonable working conditions. They benefit from clear product requirements, appropriate technical architecture that avoids unnecessary complexity, access to necessary tools and resources, and recognition for their contributions. The development team's satisfaction directly impacts product quality and velocity, making their needs strategic considerations rather than mere operational concerns. Research by Forsgren et al. (2018) demonstrates strong correlations between developer experience and software delivery performance, suggesting that organizational practices supporting developer effectiveness ultimately benefit all stakeholders through improved product outcomes.

2.2.3 Payment Service Providers

Stripe, PayPal, and other payment infrastructure partners have financial interests in transaction volume processed through their platforms and reputational interests in ensuring all transactions comply with regulatory requirements. They benefit from integration patterns that reduce fraud risk, clear communication about transaction purposes that minimize disputes, and prompt handling of any compliance issues. Maintaining positive relationships with these partners ensures continued access to payment infrastructure and potentially preferred commercial terms as transaction volumes scale. Additionally, well-integrated implementations that follow provider best practices may receive preferential technical support and early access to new features.

2.2.3 Investors and Stakeholders with Financial Interest

If Split-sync receives external funding, investors bring financial return expectations that influence growth targets, revenue generation priorities, and exit strategies. Their goals typically include rapid user growth, clear path to profitability or acquisition, strong competitive positioning, and appropriate risk management. Balancing investor return expectations with user experience priorities and sustainable development practices requires ongoing dialogue and transparent communication about trade-offs. Blank (2013) emphasizes that successful startups distinguish between optimization for immediate investor returns versus building long-term sustainable businesses, particularly in markets requiring significant trust such as financial services.

2.2.4 Regulatory Bodies

Financial regulators in operating jurisdictions have statutory responsibilities for consumer protection, fraud prevention, anti-money laundering enforcement, and market stability. While Split-sync avoids direct financial institution licensing by partnering with licensed payment providers rather than holding user funds, regulators still maintain interest in ensuring that the coordination layer doesn't facilitate illicit activity or expose consumers to unreasonable risks. Proactive compliance with applicable regulations and transparent cooperation with regulatory inquiries supports long-term operational viability. Voigt and Von dem Bussche (2017) argue that

treating regulatory compliance as strategic opportunity rather than burdensome obligation differentiates companies that thrive in regulated markets from those that struggle.

2.2.5 Partner Organizations

Services that integrate with Split-sync such as travel platforms, housing marketplaces, or accounting tools have interests in providing value to their users through the integration while protecting their own user experience and data security. They need clear documentation of integration capabilities, responsive technical support, and confidence that Split-sync will remain operational and maintain stable APIs. Successful partnership relationships require ongoing communication about roadmap changes and mutual investment in supporting shared users.

2.2.6 Broader Society and Community

Beyond specific stakeholder groups, Split-sync operates within broader societal context that includes impacts on financial literacy, social relationships, and economic behavior. While these impacts may seem abstract compared to direct user needs, they inform design choices particularly around features that could enable harmful behaviors such as excessive tracking of partners' spending or social pressure around repayment. Design ethics literature (Friedman and Hendry, 2019) emphasizes that technologists bear responsibility for foreseeable consequences of their work, suggesting that considering societal impacts represents not merely altruism but professional obligation.

2.3 Split-sync Context Diagram

The context diagram illustrates how Split-sync fits within its broader technical ecosystem, identifying external systems it interacts with and the nature of those interactions. Understanding these relationships informs architectural decisions and reveals dependencies that impact system reliability and performance.

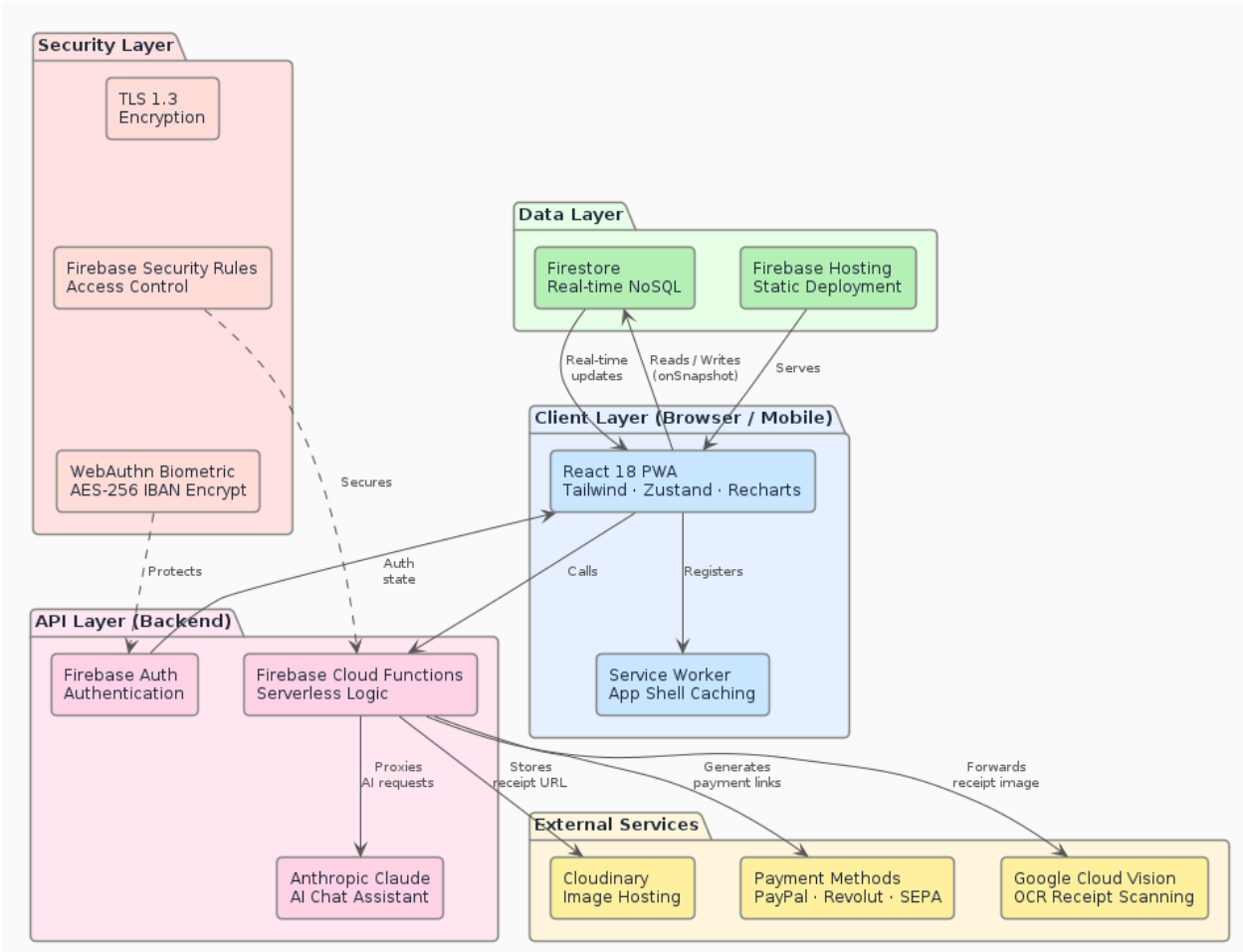


Fig. 2 Context diagram for Split-Sync

2.3.1 Group Members (External Actors)

Friends, roommates, couples, and travel teams interact with Split-sync through web browsers on mobile devices or desktop computers, accessing the progressive web application over HTTPS connections. These users generate all input data including expense details, receipt images, split preferences, settlement initiations, and group management actions. They receive outputs including balance calculations, payment suggestions, transaction history, and notifications about group activity. The relationship is fundamentally bidirectional, with users both consuming information and contributing data that creates value for their group members.

2.3.2 Web App + API (Node + Firebase)

The core application system comprises the user-facing React-based progressive web application and the backend API implemented through Node.js running in Firebase Cloud Functions. This combined system handles user authentication, data storage and retrieval, business logic execution, payment processing coordination, and notification generation. The system maintains the authoritative ledger of all expenses, splits, and settlements, ensuring data consistency across group members. Firebase's real-time database capabilities enable live synchronization of balance updates without requiring users to manually refresh.

2.3.3 OCR Engine

External optical character recognition services process receipt images uploaded by users, extracting structured data such as merchant names, dates, amounts, and line items. Split-sync sends image data to OCR providers and receives back parsed text and confidence scores. Depending on implementation choices, this may involve on-premises Tesseract processing, cloud-based services such as Google Vision AI, or specialized financial document OCR services such as Taggun or Klippa. The system validates OCR results against heuristics such as expected numerical formats and presents extracted data to users for confirmation before committing to the ledger, implementing a human-in-the-loop pattern that improves accuracy while maintaining convenience (Kumar and Zhang, 2023).

2.3.4 Cloud Storage (Receipts)

Firebase Storage and optionally Cloudinary host receipt images and PDF documents uploaded by users. The storage system maintains access control ensuring that only members of relevant groups can retrieve images, encrypts data at rest for privacy protection, and provides redundant storage to prevent data loss. Image hosting services also optimize delivery through content delivery networks that serve images from geographically distributed servers, reducing load times for users accessing receipts from their transaction history. According to Grigorik (2013), effective CDN usage can reduce image load times by 50-70% for international users compared to single-region hosting.

2.3.5 Web Push (Push Service)

Browser push notification services maintained by browser vendors and operating system providers enable Split-sync to send notifications to users even when they don't have the application open. The system sends notification requests to push service providers including information about the notification recipient and content, and the provider handles delivery to the appropriate device. This architecture follows the Web Push Protocol standardized by the IETF, ensuring compatibility across browsers and platforms. The push service acts as trusted intermediary, validating that notification requests come from authorized sources before delivering to users.

2.3.6 Payment GW (Stripe/PayPal/Revolut/SEPA)

Payment gateway providers process financial transactions initiated through Split-sync, handling credit card charges, bank transfers, digital wallet payments, and account-to-account transfers through Open Banking protocols. Split-sync sends payment requests to gateway APIs including amount, currency, recipient details, and metadata linking transactions to specific settlements. The gateway executes payment processing including fraud screening, Strong Customer Authentication when required by regulation, and international currency conversion when necessary. Upon completion, the gateway sends webhook notifications back to Split-sync indicating success or failure, which the system uses to update settlement records.

2.3.7 Bank/Card (Account to Account)

The ultimate source and destination of funds involved in settlements, users' bank accounts and credit cards connect to payment gateways through established financial networks. Split-sync never directly interacts with bank accounts or card networks, maintaining bank agnosticism by operating exclusively through payment gateway abstraction layers. This architecture simplifies compliance requirements while supporting diverse payment methods that users prefer. The bank/card systems send payment confirmation events to gateways, which forward relevant information to Split-sync through webhooks, creating a clear audit trail of settlement processing.

2.3.8 Network Connections

All communication between system components occurs over authenticated, encrypted network connections following industry-standard security protocols. HTTPS protects data in transit between users and the Split-sync application, mutual authentication ensures that API requests originate from legitimate sources, and webhook signature verification confirms that incoming payment notifications actually come from payment gateways rather than spoofing attempts. The context diagram illustrates that Split-sync acts as coordinator and data keeper rather than direct financial actor, routing payment instructions to specialized providers while maintaining application state.

The interaction patterns visible in the context diagram reveal several important architectural characteristics. Split-sync depends on external services for critical functionality, introducing potential failure modes when those services experience outages or degraded performance. The design therefore includes graceful degradation strategies such as queueing OCR processing when recognition services are unavailable and allowing manual expense entry when automatic extraction fails. The system's role as coordinator rather than direct payment processor reduces regulatory burden while requiring careful handling of payment callbacks to ensure ledger accuracy. The append-only ledger architecture simplifies synchronization and conflict resolution, as new events can be appended to the ledger without complex merge strategies required by systems allowing retroactive modifications (Kleppmann, 2017).

3.0 Functional Specifications

3.1 Use Case Model

The use case model defines the functional requirements of Split-sync by identifying the actors who interact with the system and the specific use cases that represent discrete units of functionality delivering value to those actors. This model forms the foundation for both development planning and testing strategy, ensuring that all essential capabilities are identified and prioritized appropriately.

3.1.1 Actors

User: Represents any authenticated individual using Split-sync, with capabilities spanning group creation, expense addition, split management, balance review, and settlement processing. Users take primary responsibility for generating the data that creates value within the system. They initiate most interactions with the application, responding to notifications and addressing outstanding balances based on their financial management preferences. The User actor encompasses diverse technical sophistication levels and engagement patterns, from power users who leverage advanced features to occasional participants who primarily view notifications and settle debts.

Group Member: While technically a specialized role of the User actor, distinguishing Group Members emphasizes the collaborative nature of expense sharing. Group Members participate in shared expenses by being assigned split portions, viewing group balances, and receiving notifications about relevant expenses. Not all Group Members actively add expenses; some may participate passively, only engaging when settling their portions. This distinction matters for notification strategies and permission models, as active expense adders require different interface designs than passive participants.

System Admin: Administrative personnel responsible for backend monitoring, user support escalations, data integrity maintenance, and infrastructure management operate with elevated

privileges not available to regular users. System Admins analyze usage patterns to inform feature development, investigate payment processing anomalies, respond to security incidents, and maintain operational systems. While users rarely interact directly with System Admins, their behind-the-scenes activities ensure reliable platform operation.

Payment Gateway: External payment service providers act as system actors when they send webhook notifications about payment status changes. From the system's perspective, payment gateways initiate certain interactions rather than merely responding to requests, particularly when asynchronous payment processes complete after initial requests. Modeling payment gateways as actors emphasizes the event-driven architecture of settlement processing.

OCR Service: Similar to payment gateways, OCR providers function as actors when they return processing results for uploaded receipts. The asynchronous nature of OCR processing means that services deliver results through callbacks rather than synchronous responses, influencing how the system manages expense creation workflows.

3.1.2 Use Cases

The following use cases represent the discrete functional capabilities that Split-sync must provide to support its value propositions. Each use case describes a complete interaction sequence between actors and the system, resulting in observable outcomes that deliver value.

3.1.2.1 Use Case 1: User Registration and Authentication

This involves new users creating accounts with email addresses and passwords, then using those credentials to access the application in future sessions. This foundational use case enables all other functionality by establishing user identity and session management. Security considerations include password strength requirements, secure credential storage using industry-standard hashing algorithms, and protection against brute-force authentication attempts. Future iterations plan to incorporate passkey authentication following WebAuthn standards for improved security and usability (Lyastani et al., 2020).

3.1.2.2 Use Case 2: Create Group and Invite Members

This allows users to establish new expense-sharing contexts by defining group names, default currencies, and initial membership. Users generate invitation links or QR codes that prospective members scan to join. The use case must handle various edge cases including expired invitations, users who lack existing accounts, duplicate join attempts, and group capacity limits. Group creators receive owner roles with full administrative privileges, distinguishing them from members invited subsequently.

3.1.2.3 Use Case 3: Add Expense with Receipt Scan

This represents the primary data input mechanism, enabling users to document shared costs by photographing receipts, uploading images, or manually entering details. The system extracts structured information through OCR when images are provided, presenting parsed data to users for verification before committing to the ledger. This use case must gracefully handle various image quality levels, receipt formats from diverse merchants, and occasional OCR failures. According to Smith and Zhang (2022), even sophisticated OCR systems benefit from human verification for financial documents where accuracy is critical.

3.1.2.4 Use Case 4: Split Expense Equally or Custom

This follows expense addition, allowing users to specify how costs divide among group members through equal division, percentage allocation, weighted shares, or per-item assignment. The system validates that specified splits sum correctly to the total expense amount, flagging discrepancies for user correction. Split preferences can be established as defaults for future expenses, reducing repetitive data entry for groups with consistent patterns such as couples who always split equally.

3.1.2.5 Use Case 5: View Balance Ledger

This provides users with the current state of who owes whom within each of their groups. The ledger displays net balances rather than individual transactions, aggregating all expenses and payments into simplified debt relationships. Users can expand summary views to see the underlying transactions that contribute to specific balances, maintaining transparency about

calculation methods. Real-time synchronization ensures that when one group member adds an expense, other members see updated balances within seconds without manual refreshing.

3.1.2.6 Use Case 6: Settle Debts via Payment Gateway

This enables users to process actual money transfers that resolve outstanding balances shown in the ledger. Users select suggested payments generated by the optimization algorithm, choose payment methods such as cards or bank transfers, and authorize transactions through payment gateway interfaces. The system monitors payment status through webhooks and automatically updates the ledger when settlements complete successfully, requiring no manual recording by either paying or receiving party. Failed payments trigger notifications and remain in the ledger as pending settlements until successfully processed or explicitly cancelled.

3.1.2.7 Use Case 7: View Transaction History

This presents users with chronological timelines of all expense additions and settlements within specific groups. The history maintains immutable records preventing retroactive modifications while allowing users to add notes or categorizations to existing entries. Users can filter history by date ranges, participants, amount thresholds, or expense categories to locate specific transactions or analyze spending patterns. CSV export functionality allows users to download complete histories for external analysis or record-keeping.

3.1.2.8 Use Case 8: Receive Notifications

This involves the system proactively informing users about events requiring attention or awareness, including new expenses added to groups they participate in, settlements completed by other members, approaching expense milestones such as trip total reaching budgets, and reminders about outstanding balances. Users configure notification preferences per group, selecting which events trigger notifications and through which channels such as push notifications or email. Research by Okoshi et al. (2017) demonstrates that notification personalization significantly impacts whether users perceive alerts as helpful versus intrusive.

3.1.2.9 Use Case 9: Offline Expense Entry

This allows users to document expenses in locations lacking reliable connectivity by queuing operations locally with automatic synchronization when network access resumes. The system stores pending actions in browser IndexedDB, maintaining operation order and conflict resolution strategies. Background Sync API integration ensures that queued operations process even if users close the application before manually reconnecting. This use case proves particularly valuable during travel scenarios where expense capture often occurs in areas with poor mobile signal.

3.1.2.10 Use Case 10: Visualize Debt Graph

This provides graphical representation of group financial relationships using nodes for members and edges for debts, with visual encoding of amount magnitudes through line thickness and direction through arrows. Interactive elements allow users to tap graph edges to view underlying transactions or initiate settlements for specific debts. The graph dynamically updates as expenses are added or settlements processed, providing immediate visual feedback on how actions affect group balance topology. According to Ware (2012), graph-based representations of network structures enhance pattern recognition and comprehension compared to tabular or list-based alternatives.

The use case model reveals several important architectural and design implications. The prevalence of user-initiated actions suggests that the application follows a reactive architecture where system behavior primarily responds to user requests rather than autonomous background processing, simplifying reasoning about system state and causality. The requirement for real-time updates across multiple users indicates the need for websocket or similar persistent connection technologies that enable server-initiated data pushes. Dependencies between use cases, such as expense splitting requiring prior expense addition, inform workflow design and validation requirements. Finally, the emphasis on group-based interactions rather than individual user actions highlights the collaborative nature of the application, suggesting that notification strategies and permission models must consider multi-user coordination.

3.2 Use Cases Diagram

This section provides concise descriptions of each use case identified in the model, establishing clear understanding of purpose, actors, preconditions, success criteria, and primary flow without detailed exception handling or alternative paths. Below is the use case diagram.

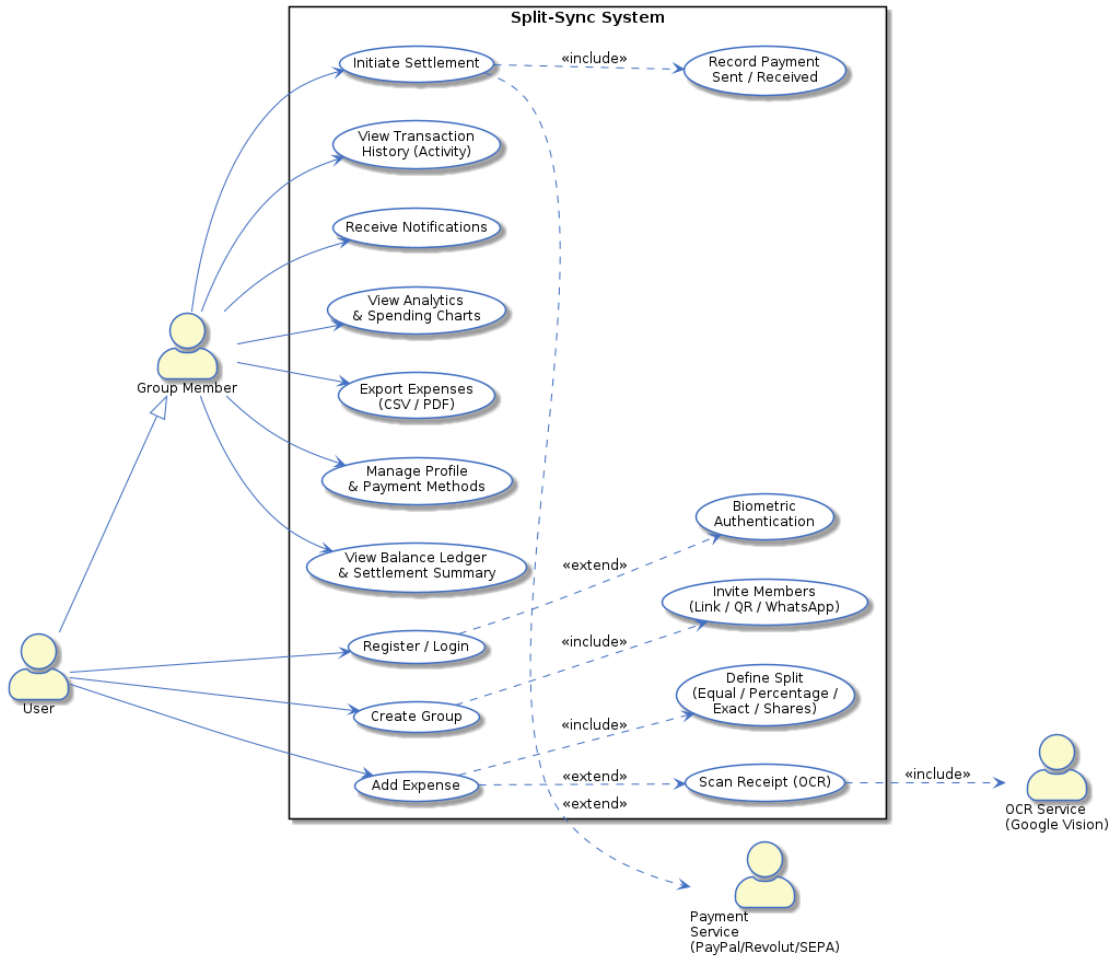


Fig. 3 Use case diagram for Split-sync

3.2.1 UC-01: User Registration and Authentication

Primary Actor: New User or Returning User

Stakeholders: User, Firebase Authentication Service

Preconditions:

- User has an internet connection
- User has a valid email address
- User has access to a modern web browser

Success Guarantee: User possesses an authenticated session and is redirected to the Dashboard with access to all application features.

Main Success Scenario — Registration:

1. User navigates to the Split-Sync login page at splitsync-dev.web.app
2. System displays the login form with options to sign in or register
3. User taps Register
4. System displays the registration form requesting full name, email address, password, and password confirmation
5. User enters their details — password must be at least 8 characters
6. System displays a real-time password strength indicator via the PasswordStrength component
7. System validates: passwords match, password meets the minimum length, and email format is valid
8. System calls Firebase Authentication `createUserWithEmailAndPassword`
9. Firebase Authentication creates the account — password hashing is handled entirely by Firebase Authentication server-side
10. System calls `updateProfile` to set the user's display name in Firebase Auth
11. System creates a user profile document in Firestore with display name, email, photo URL, default preferences, notification settings, and `isFirstLogin: true`
12. User is redirected to the Dashboard

13. System detects `isFirstLogin: true` and launches the Onboarding Wizard to guide the user through initial setup

Alternative Flow UC-01b: Email and Password Login

1. User navigates to the login page
2. User enters their email and password and taps Sign In
3. System calls Firebase Authentication `signInWithEmailAndPassword`
4. On success, system checks whether biometric authentication is enabled for this user on this device
5. If biometric is enabled, system calls `authenticateWithBiometric` using the WebAuthn API — user provides fingerprint or Face ID
6. If biometric passes, user is redirected to the Dashboard
7. If biometric fails, system signs the user out and displays an error — biometric authentication is required on devices where it has been enabled
8. If biometric is not enabled on this device, user is redirected to the Dashboard immediately after password verification

Alternative Flow UC-01c: Google OAuth Sign-In

1. User taps Continue with Google on the login page
2. System calls `signInWithPopup` using `GoogleAuthProvider`
3. Firebase Authentication handles the Google OAuth flow
4. On success, system checks whether this is a new or existing user
5. If new user: system creates a Firestore profile document using the Google account name and photo, sets `isFirstLogin: true`, and redirects to Dashboard where the Onboarding Wizard launches

6. If existing user: system updates the profile photo from Google if no custom photo has been uploaded, and redirects to Dashboard

Alternative Flow UC-01d: Forgot Password

1. User taps Forgot Password on the login page
2. System navigates to the ForgotPassword page
3. User enters their email address
4. System calls Firebase Authentication `sendPasswordResetEmail`
5. Firebase sends a password reset email to the address
6. User clicks the reset link in the email and sets a new password via Firebase's hosted reset page

Extensions:

- **7a. Passwords do not match:** System displays "Passwords do not match" and blocks form submission.
- **7b. Password is fewer than 8 characters:** System displays "Password must be at least 8 characters" and blocks submission.
- **7c. Email is already registered:** Firebase returns `auth/email-already-in-use`. System displays "Email already in use" with a suggestion to sign in instead.
- **7d. Email format is invalid:** Firebase returns `auth/invalid-email`. System displays "Invalid email address."
- **7e. Password is too weak per Firebase standards:** Firebase returns `auth/weak-password`. System displays "Password is too weak."
- **5b (Login). Incorrect credentials:** Firebase returns an authentication error. System displays "Invalid email or password."

- **5c (Login). Biometric fails:** System signs the user out and displays "Biometric authentication required to login on this device."

Frequency of Use: Once per user for registration. Multiple times per user for subsequent logins.

Performance Requirements:

- Registration and Firestore profile creation: under three seconds
- Email and password login: under two seconds
- Google OAuth sign-in: under three seconds including the popup flow
- Biometric authentication prompt: under one second after password verification

Security Requirements:

- Password hashing is performed server-side by Firebase Authentication
- All authentication traffic uses HTTPS enforced by Firebase Hosting
- Biometric credentials are stored on-device using the WebAuthn API — they are never transmitted to the server
- Session state is managed by Firebase Authentication and persisted in the browser via the Zustand auth store with localStorage

3.2.2 UC-02: Create Group and Invite Members

Primary Actor: Authenticated User (Group Creator)

Stakeholders: Group Creator, Prospective Members

Preconditions:

- User is authenticated
- User is on the Dashboard page

Success Guarantee: A new group exists in Firestore with the creator as owner, and any invited members have received in-app invitations or can join via the shared invite link.

Main Success Scenario:

1. User taps the Create Group button on the Dashboard
2. System displays the CreateGroupModal with a group creation form
3. User enters a group name (required, maximum 50 characters)
4. User selects an icon from the IconSelector (defaults to Home)
5. User selects a currency from the available options: EUR (default), USD, or GBP
6. System validates: group name is not empty and does not exceed 50 characters
7. System creates the group document in Firestore with name, icon, currency, createdBy, memberIds array, and members array containing the creator with role owner
8. System sends in-app invitations to any members pre-selected during group creation via Quick Add
9. System displays the group page — the creator is now the group owner
10. User taps Invite Link in the group to open the ShareInviteModal
11. System generates a shareable invite URL in the format `/invite/{groupId}`
12. System displays the invite link with options to copy, share via WhatsApp, share via SMS, share via Email, or show as a QR code
13. User shares the link via their preferred method
14. Prospective member receives the link and taps it
15. System opens the JoinGroup page
16. If the member is not yet registered, system redirects them to the Register page and after registration redirects back to the invite link

17. If the member is already authenticated, system validates the invitation — checks it exists, is not expired, and the user is not already a member
18. System adds the member to the group's memberIds and members arrays
19. System updates the invitation status to accepted
20. System notifies the group creator via an in-app notification that a new member has joined
21. The group member list in GroupDetails updates in real time via onSnapshot for all current members

Invitation Methods Support:

Method	How It Works
Copy Link	Copies the invite URL to the clipboard
WhatsApp	Opens WhatsApp with a pre-filled message containing the link
SMS	Opens the device SMS app with a pre-filled message
Email	Opens the device email client with a pre-filled subject and body
QR Code	Displays a scannable QR code generated from the invite URL
Quick Add from Connections	Pre-selects known contacts from previous groups during group creation

Extensions:

- **6a. Group name is empty:** System displays a validation error and blocks saving. User must enter a name before proceeding.
- **6b. Group name exceeds 50 characters:** System displays a validation error. The input field also enforces maxLength={50} to prevent typing beyond the limit.
- **17a. Invitation has expired (older than 7 days):** System displays an expiry message on the JoinGroup page. The invitation status is updated to expired in Firestore.
- **17b. User is already a member of the group:** System displays a message that they are already in the group and navigates them to the group page.
- **17c. Invitation does not exist or has been cancelled:** System displays an invalid invitation message.
- **21a. Creator has the group page open when the member joins:** The members list updates in real time without requiring a refresh, via the onSnapshot listener on the group document.

Frequency of Use: Occasional — when starting a new shared expense context such as a trip, household, or event.

Performance Requirements:

- Group creation in Firestore: under two seconds
- Invite link generation: instant — the URL is derived from the groupId
- QR code generation: under 500ms
- Member join and group update propagation: real time via onSnapshot

Business Rules:

- Group name is required and maximum 50 characters
- Creator is automatically assigned the owner role and cannot be removed
- Invite links are valid for 7 days from creation

- A user cannot join a group they are already a member of
- A user cannot join a group using an expired or cancelled invitation

3.2.3 UC-03: Add Expense with Receipt Scan

Primary Actor: Group Member

Stakeholders: All Group Members, Google Cloud Vision API (OCR Service), Cloudinary (Image Storage)

Preconditions:

- User is authenticated and a member of at least one group
- User is inside the AddExpenseModal within a group

Success Guarantee: Receipt image is uploaded to Cloudinary, OCR extracts the merchant name, total amount, and date, and the extracted data pre-fills the expense form for user review and correction before saving.

Main Success Scenario:

1. User opens the AddExpenseModal from within a group
2. User taps the Scan Receipt icon button in the modal
3. System opens the ReceiptScanner component as an overlay modal
4. System presents two input options: Scan Receipt with Camera, or Upload from Gallery
5. User selects Scan Receipt with Camera — system triggers the device camera using an HTML file input with `capture="environment"`
6. User captures the receipt image
7. System displays a preview of the selected image
8. User taps Process Receipt
9. System uploads the image to Cloudinary and stores the returned secure URL
10. System converts the image to base64 and sends it to the `scanReceipt` Firebase Cloud Function

11. Cloud Function forwards the base64 image to the Google Cloud Vision API for text detection
12. Google Cloud Vision extracts the full text from the receipt
13. Cloud Function parses the extracted text to identify the merchant name (first meaningful text line), the total amount (highest decimal value found or pattern-matched from keywords such as "total" or "amount"), and the date
14. System displays the Review and Edit screen with three pre-filled editable fields: Description or Merchant Name, Total Amount, and Date
15. User reviews the extracted data, corrects any errors, and taps Use This Data
16. System passes the extracted data back to the AddExpenseModal, pre-filling the title, amount, date, and receipt URL fields
17. User completes the remaining expense fields (category, split method, participants) and saves

Alternative Flow UC-03b: Upload from Gallery

1. At step 4, user selects Upload from Gallery instead of Camera
2. System triggers an HTML file input without the capture attribute
3. User selects an image from their photo library
4. Continue from step 7 of the main scenario

Alternative Flow UC-03c: Manual Entry

1. User closes the ReceiptScanner without scanning
2. User fills in all expense fields manually in the AddExpenseModal
3. No receipt image is attached

Extensions:

- **9a. Cloudinary upload fails:** System logs the error but continues with OCR processing. The receipt URL is not stored — expense can still be saved without the image.
- **11a. Cloud Function call fails or times out:** System displays an alert with the error message. User is returned to the scanner screen and can retry or close and enter manually.

- **13a. OCR extracts no recognisable amount:** The amount field is returned empty. User must enter the amount manually in the review screen before proceeding.
- **13b. OCR extracts no recognisable date:** Date defaults to today's date. User can correct it in the review screen.
- **15a. User leaves required fields blank:** System validates on submit — title and a valid amount greater than zero are required before the data is passed back to the modal.

Frequency of Use: Optional — available for every expense entry but not required.

Performance Requirements:

- Cloudinary image upload: under five seconds on a standard mobile connection
- Google Cloud Vision OCR processing: under ten seconds
- Review screen display after processing: immediate

3.2.4 UC-04: Split Expense

Primary Actor: Group Member (Expense Creator)

Stakeholders: All Group Members

Preconditions:

- User is adding an expense within a group (UC-03 in progress)
- Group has at least two members
- Expense amount is valid, positive, and does not exceed 1,000,000 in the group currency

Success Guarantee: Expense is correctly divided among selected participants, the Firestore expense record is created, balances update in real time, and all group members are notified.

Main Success Scenario:

1. Within the AddExpenseModal, system displays all group members with checkboxes — all selected by default
2. User reviews and adjusts the participant list by checking or unchecking members
3. System displays four split method options as selectable tabs: Equal Split (default), By Percentage, Exact Amounts, By Shares
4. User selects a split method
5. System immediately recalculates and displays the resulting amount per participant based on the selected method
6. User reviews the per-person breakdown shown below each member's name
7. User clicks Save Expense
8. System validates the split — method-specific rules are enforced before saving
9. System creates the expense document in Firestore with the title, amount, currency, category, date, paidBy, participants array, splits map, and splitMethod
10. System sends an in-app notification to every group member informing them of the new expense, including the amount and their individual share
11. System closes the modal and the group expense list updates in real time via the onSnapshot listener.

Split Methods Implemented:

Method	How It Works	Validation Rule
Equal Split	Total divided evenly across all selected participants	No manual input required — calculated automatically
By Percentage	User enters a percentage for each selected participant	Percentages must sum to exactly 100% — tolerance 0.01%
Exact Amounts	User enters a precise monetary amount per participant	Amounts must sum to the total expense — tolerance €0.01
By Shares	User assigns a share count per participant (e.g. 2 vs 1)	System calculates proportional amounts — no manual total check

Extensions:

- **2a. User deselects a participant:** System removes them from the split calculation and recalculates shares among the remaining selected members in real time.
- **2b. No participants selected:** System displays an error: "Select at least one member" and blocks saving.
- **5a. Equal Split selected:** System automatically divides the total evenly. No manual input is needed. Rounding is handled by distributing the remainder to the last participant.
- **5b. By Percentage selected:** System displays a percentage input per selected member. A running total shows how close the sum is to 100%. Saving is blocked until exactly 100% is reached.

- **5c. Exact Amounts selected:** System displays an amount input per selected member. A running total shows the remaining amount to allocate. Saving is blocked if the sum does not equal the total within €0.01.
- **5d. By Shares selected:** System displays a share count input per member defaulting to 1. System calculates each person's amount proportionally. No manual sum check is required.
- **8a. Validation fails:** System displays a specific inline error message and blocks saving until the issue is resolved.
- **10a. Notification fails for one or more members:** The expense is saved successfully regardless. Notification failure is logged to the console but does not block the save.
- **Editing an existing expense:** AddExpenseModal supports pre-population of all fields when opened in edit mode. The same split method and participant selection is restored and the user can modify any field before saving.

Frequency of Use: Every expense addition — very frequent.

Business Rules:

- Expense creator is included in the split by default but can be deselected
- Split amounts are stored to two decimal places
- Maximum expense amount is 1,000,000 in the group currency
- Individual split amounts cannot be zero
- Expense title is required — saving is blocked without it

Performance Requirements:

- Split recalculation on input change: instant — no network call required
- Expense save to Firestore: under two seconds
- Group expense list update after save: real time via onSnapshot

3.2.5 UC-05: View Balance Ledger

Primary Actor: Group Member

Stakeholders: All Group Members

Preconditions:

- User is authenticated
- User is a member of at least one group
- Group contains at least one expense

Success Guarantee: User views accurate, current balance information for their groups and can initiate settlement from the balance view.

Main Success Scenario:

1. User navigates to a group in GroupDetails
2. System calculates current net balances in real time using the `calculateBalances` function applied to all expenses and settlements in the group
3. System applies the greedy two-pointer minimum cash-flow algorithm via `calculateSettlements` to determine the optimal settlement paths
4. System displays a Balance Summary card at the top of the group page showing:
    - Whether the user is owed money, owes money, or is fully settled
    - The net amount in the group's currency, colour-coded — blue if owed, orange if owing, green if settled
5. System displays the Settlement Summary section listing each required payment — who pays whom and how much — using member profile photos or initials
6. Each settlement row shows a Settle Up button for involved members, and a Mark as Received button for the payee
7. User taps Settle Up to initiate payment via UC-06
8. System also displays a Full Balance Breakdown section listing the raw net balance for every group member

9. User can also access per-person balance detail via the BalanceBreakdownModal from the Profile page, which shows which groups contribute to each person's balance

Additionally on the Profile page:

1. User navigates to the Profile (You) tab
2. System aggregates balances across all groups, grouped by currency
3. System displays Owed to You and You Owe cards — each showing separate lines per currency with no cross-currency summation
4. User taps either card to open the BalanceBreakdownModal
5. System displays a per-person breakdown — each person shown with the amounts owed per currency and which groups contribute, with a clickable link to navigate directly to that group

Extensions:

- **2a. Group has no expenses yet:** System shows a "No expenses yet" empty state with an Add Expense call-to-action.
- **4a. User is fully settled in the group:** Balance Summary card shows a green All Settled Up state with a checkmark icon.
- **6a. Payment confirmation is pending:** A sticky amber banner appears at the top of GroupDetails prompting the payee to Confirm Received or Decline, and prompting the payer to Mark as Paid if they completed the external payment.
- **7a. Payee has no payment methods saved:** The Settle Up modal displays a message that the payee has not configured payment details and offers the Manual / Paid Offline option.
- **9a. User views Profile balance breakdown:** BalanceBreakdownModal shows all people who owe the user or whom the user owes, with amounts grouped per currency and clickable group rows to navigate to that group for settlement.

Frequency of Use: Very frequent — checked before and after adding expenses and completing settlements.

Performance Requirements:

- Balance calculation: real time, recalculated on every Firestore onSnapshot update
- Settlement Summary rendering: under 500ms
- BalanceBreakdownModal load: under 300ms
- Real-time sync: under 500ms from remote write to local UI update via onSnapshot

Business Rules:

- Balances are always displayed in the group's currency — no cross-currency conversion is applied
- Zero balances are not shown in the Settlement Summary
- The settlement algorithm minimises the number of transactions required to clear all debts in a group
- Balance amounts are accurate to two decimal places

3.2.6 UC-06: Settle Debts via Payment Methods

Primary Actor: Group Member (Payer)

Stakeholders: Payee, System

Preconditions:

- User is authenticated
- User has an outstanding debt in a group (negative balance)
- Payee has at least one payment method saved in their profile (PayPal username, Revolut username, or encrypted IBAN)

Success Guarantee: Payment is initiated or recorded, both parties are notified via the in-app confirmation flow, and balances update once the payee confirms receipt.

Main Success Scenario:

1. User opens a group and views the Settlement Summary showing who owes who
2. User taps the Settle Up button next to a settlement

3. System opens the SettleUpModal displaying the settlement amount, payer and payee names, and the group context
4. System fetches the payee's saved payment methods from their Firestore profile
5. System displays the available payment options based on what the payee has configured
6. User reviews the amount — the amount field is editable to allow partial payment
7. User selects a payment method and initiates payment:
    - **PayPal:** System generates a PayPal.me direct payment link pre-filled with the amount and currency. User taps the link which opens PayPal externally. Alternatively, user can pay via the embedded PayPal button which processes payment in-app.
    - **Revolut:** System generates a Revolut payment link pre-filled with the amount. User taps the link which opens Revolut externally.
    - **SEPA Bank Transfer:** System decrypts the payee's IBAN and displays bank transfer details (IBAN, BIC if available, amount, reference). User completes the transfer via their own banking app.
    - **Manual / Paid Offline:** User records that payment was made externally (cash, bank transfer, or other method not listed).
8. After initiating payment, user taps the confirmation button in the modal
9. System calls `recordPaymentSent` which writes two Firestore notification documents: a `payment_confirmation_needed` notification to the payee and a `payment_log` record for the payer
10. Payee receives an in-app notification and is prompted to confirm or decline receipt
11. If payee confirms: system updates the payment log status to confirmed, writes the settlement as an expense record in Firestore, and balances recalculate in real time
12. If payee declines: system updates the payment log status to declined and the payer is notified.

Payment Methods Supported:

Method	How it Works	Confirmation Required
PayPal (link)	Generates a PayPal.me URL pre-filled with amount	Yes — payee confirms in app
PayPal (button)	Embedded PayPal SDK processes payment in-app	Yes — payee confirms in app
Revolut	Generates a Revolut payment URL	Yes — payee confirms in app
SEPA Bank Transfer	Displays IBAN and transfer details for user to action manually	Yes — payee confirms in app
Manual / Paid Offline	Records payment without a payment link	Yes — payee confirms in app

Extensions:

- **6a. Payee has no payment methods saved:** System displays a message that the payee has not set up payment methods. User can still use the Manual / Paid Offline option.
- **6b. User edits the amount:** User can reduce the amount to make a partial payment. The remaining balance stays in the ledger after confirmation.

- **7a. PayPal link opens externally:** User completes payment in the PayPal app or website, then returns to Split-Sync and taps the confirmation button to record that payment was sent.
- **10a. Payee has not confirmed after payment:** The payment log shows status as Awaiting Confirmation. A sticky banner appears in GroupDetails prompting the payee to confirm or decline.
- **11a. Payee declines:** Payer is notified in-app. The debt remains in the ledger and the user can attempt settlement again.
- **12a. Biometric authentication required:** Before viewing or editing payment settings, the user must authenticate with biometric or password.

Frequency of Use: Regular — weekly to monthly depending on group activity.

Performance Requirements:

- SettleUpModal load: under two seconds
- Payment link generation: instant
- Payee notification delivery: under 500ms via Firestore onSnapshot
- Balance recalculation after confirmation: real time

Security Requirements:

- Payee IBAN is stored encrypted with AES-256 and decrypted only at point of display within the authenticated modal
- Payment settings require biometric or password authentication before viewing or editing
- No raw card details are entered or stored at any point in the application

3.2.7 UC-07: View Transaction History

Primary Actor: Group Member

Stakeholders: All Group Members

Preconditions:

- User is authenticated
- User is a member of at least one group
- At least one expense or settlement exists in the user's groups

Success Guarantee: User views an accurate, filterable list of all expenses and settlements across their groups, and can view full details, edit, delete, or export entries where permitted.

Main Success Scenario:

1. User navigates to the Activity page from the bottom navigation bar
2. System fetches all expenses from all groups the user belongs to via real-time onSnapshot listeners
3. System sorts all expenses by date, newest first
4. System displays summary stat cards at the top showing total paid and total owed, each broken down per currency
5. System displays the full expense list with each item showing: title, group name, date, total amount with currency, and whether the user paid or owes
6. User taps a filter tab — All, I Paid, or I Owe — to narrow the list
7. User taps an expense to open the Expense Details modal
8. System displays full expense details including: title, category, date, total amount, split breakdown per member, description or notes if present, and receipt image if one was scanned

9. User returns to the activity list

Alternatively, within a specific group:

1. User navigates to a group in GroupDetails
2. User scrolls to the Recent Expenses section
3. User taps an expense to open the Expense Details modal
4. If the user created the expense and no settlements exist in the group, Edit and Delete options are available
5. User taps Edit — system opens the AddExpenseModal pre-filled with the existing expense data
6. User modifies details and saves — system updates Firestore and recalculates balances
7. User taps Delete — system shows a confirmation dialog and permanently removes the expense from Firestore, recalculating balances for all members
8. User exports all expenses from the group via the three-dot menu — Export CSV or Export PDF

Filtering Implemented:

Filter	Description
All	Shows all expenses across all groups
I Paid	Shows only expenses where the user is the payer

I Owe	Shows only expenses where the user is a participant but not the payer

Export Options Implementation (per group, from GroupDetails):

Format	Contents
CSV	Date, title, category, amount, currency, paid by, your share, split method, notes
PDF	Member summary, settlement summary, full expense list — generated via browser print dialog

Extensions:

- **6a. No expenses match the selected filter:** System displays an appropriate empty state message.
- **8a. Receipt image was scanned for the expense:** Image is displayed within the Expense Details modal.
- **8b. Expense has no description or notes:** Those fields are hidden from the detail view.
- **4b (GroupDetails). Group has existing settlements:** Edit is blocked for regular expenses to preserve settlement integrity. A notice is shown explaining why editing is locked. Delete remains available for the expense creator.

- **4c (GroupDetails). User did not create the expense:** Edit and Delete options are not shown.

Frequency of Use: Regular — multiple times per week for active groups.

Performance Requirements:

- Activity page load: under two seconds for one hundred expenses
- Real-time expense updates via onSnapshot: under 500ms
- Export CSV generation: instant for groups with up to one thousand expenses
- Export PDF generation: under three seconds via browser print dialog

3.2.8 UC-08: Receive Notifications

Primary Actor: System (initiating), Group Member (receiving)

Stakeholders: Group Members

Preconditions:

- User is authenticated and a member of at least one active group
- Notification subscription is active via the Zustand notifications store

Success Guarantee: User receives a timely, relevant in-app notification for group activity and can act on it directly from the notification centre.

Main Success Scenario:

1. System detects a notification-worthy event — a new expense is added to a group the user belongs to, a payment is sent to the user, a payment is confirmed or declined, or a group invitation is received
2. System writes a notification document to the Firestore notifications collection, including

type, title, message, read status, and any relevant metadata

3. The active onSnapshot listener in the notifications store receives the new document in real time
4. The notification bell badge count increments to reflect the number of unread notifications
5. User opens the notifications dropdown from any page in the app
6. System displays notifications in reverse chronological order with unread items visually distinguished
7. User taps a notification or its inline action button
8. System marks the notification as read and decrements the badge count
9. For actionable notifications, the user confirms or declines directly from the notification

Notification Types Implemented:

Type	Trigger	Requires Action
new_expense	A new expense is added to a group the user belongs to	No
group_invitation	The user is invited to join a group	Yes — Accept or Decline
payment_confirmation_needed	Another user claims to have paid the user	Yes — Confirm or Decline

payment_log	The user sent a payment — personal status record	No

Extensions:

- 4a. All notifications already read: Badge count remains at zero. Dropdown shows all past notifications.
- 6a. No notifications exist: Dropdown displays an empty state message.
- 8a. User marks all as read: System batch-updates all unread notifications to read status and resets badge to zero.
- 8b. User clears all notifications: System deletes all notification documents for the user from Firestore.
- 9a. Payment confirmation — user confirms: System updates the payment log status to confirmed and notifies the payer that receipt was confirmed.
- 9b. Payment confirmation — user declines: System updates the payment log status to declined and notifies the payer that the payment was not confirmed.

Frequency of Use: Frequent — multiple times per day in active groups.

Performance Requirements:

- Notification delivery via Firestore onSnapshot: under 500ms from write to display
- Badge count update: real time
- Notification dropdown load: under 300ms

3.2.9 UC-09: Offline Support (PWA Caching)

Primary Actor: Group Member

Stakeholders: System, Service Worker

Preconditions:

- User has previously loaded the app at least once while online
- Service worker is registered and active on the user's device
- App shell has been pre-cached during the initial install event

Success Guarantee: User can open the application and view the cached app shell when offline. A clear offline indication is shown and the user is informed that live data requires an internet connection.

Main Success Scenario:

1. User opens Split-Sync while offline or on an unstable connection
2. Service worker intercepts the navigation request
3. Service worker serves the cached app shell (HTML, CSS, JavaScript, icons) from the static cache
4. App loads successfully from cache without requiring a network request
5. System detects the absence of a network connection
6. System displays the offline fallback page or the last cached view with a clear offline indicator
7. User sees that real-time data such as expenses, balances, and group activity is unavailable
8. User reconnects to the internet

9. App automatically resumes normal Firestore real-time functionality via the active onSnapshot listeners

10. Data refreshes without requiring a manual page reload

Caching Strategy Implemented:

Content Type	Strategy	Details
App shell (HTML, JS, CSS, icons)	Cache-First	Pre-cached on service worker install
Navigation requests	Network-First with offline fallback	Falls back to offline.html if no cache and no network
Static assets (images, fonts)	Cache-First	Cached on first fetch, served from cache thereafter
Firebase Auth and Firestore	Network-Only	Always bypassed — never cached to prevent stale data
Cloud Function calls	Network-Only	Always bypassed — requires live network

Extensions:

- **3a. App shell not yet cached (first visit offline):** Browser has no cached content. User sees the browser's default offline error page. No fallback is available until the app has been loaded at least once while online.
- **4a. User attempts to add an expense while offline:** The AddExpenseModal requires a live Firestore connection to write data. The action fails silently or displays a network error. No local queuing is available.
- **4b. User attempts to view group details while offline:** Firestore onSnapshot listeners receive no updates. Previously rendered data may remain visible in React state until the component unmounts, but no live data is available.
- **9a. User returns to app after extended offline period:** onSnapshot listeners re-establish automatically. Firestore returns the latest data and the UI updates accordingly.

Special Requirements Met:

- App shell loads in under one second from cache when offline
- Static assets are served cache-first for improved performance on slow connections
- Firebase traffic is always network-only to prevent users seeing stale authentication or balance data

Frequency of Use: Occasional — primarily relevant when users are travelling or in areas with poor connectivity.

Business Rules:

- The app does not allow expense creation, settlement, or any Firestore write operation without an active internet connection
- All real-time data (balances, expenses, notifications) requires a live connection to Firestore

3.2.10 UC-10: Visualise Group Balances and Settlements

Primary Actor: Group Member

Stakeholders: All Group Members

Preconditions:

- User is authenticated
- User is a member of at least one group
- Group has at least one recorded expense
- At least one non-zero balance exists between members

Success Guarantee: User views an accurate visual representation of group balances and outstanding settlements across their groups.

Main Success Scenario:

1. User navigates to the Stats tab from the bottom navigation bar
2. User optionally selects a specific group from the group selector at the top of the Analytics page to filter charts to a single currency
3. System calculates current balances using the greedy two-pointer settlement algorithm
4. System renders the Per-Person Balance chart — a horizontal bar chart showing each member's net balance across all groups, coloured green if owed money and red if owing money
5. System renders the Who Owes Who section — a list of consolidated settlement suggestions showing who should pay whom and how much, each with a relative progress bar indicating the proportion of the largest settlement
6. User views the charts and settlement list to understand the financial state of their groups
7. User navigates to the relevant group and uses the Settle Up flow to action a settlement

Charts and Visualisations Implemented:

Chart	Type	Library	Description
Spending Over Time	Area chart	Recharts	Total spending trend across last 6 months
Monthly Summary	Bar chart	Recharts	Month by month spending for the current year
Category Breakdown	Pie chart	Recharts	Proportion of spending per expense category
Per-Person Balance	Horizontal bar chart	Recharts	Net balance per group member — green if owed, red if owing
Who Owes Who	Styled list with progress bars	Custom JSX	Consolidated settlement suggestions with relative proportions

Extensions:

- **2a. User selects a specific group:** All charts and totals filter to that group's currency. Monetary values display accurately in the group currency.

- **2b. User selects All Groups:** Charts aggregate data across all groups. Summary cards show per-currency stacked totals rather than a single combined figure, since currencies cannot be summed.
- **4a. No balance data exists:** Per-Person Balance chart displays an empty state message.
- **5a. All balances are settled:** Who Owes Who section displays "All settled up."
- **6a. User wants to act on a settlement:** User navigates manually to the relevant group and uses the Settle Up button in the Settlement Summary section of GroupDetails.

Frequency of Use: Moderate — weekly for active groups.

Performance Requirements:

- Balance calculation: under 500ms
- Chart rendering: under 500ms on first load
- Group selector filter: instant re-render with no additional network calls

3.3 Detailed User Stories

User stories provide detailed, user-centric descriptions of system functionality using the format "As a [type of user], I want [some goal] so that [some reason]." These stories elaborate on use cases by emphasizing user motivations and success criteria while maintaining focus on value delivered rather than implementation approaches. The following user stories cover primary features organized by functional category, with acceptance criteria defining what constitutes successful implementation.

3.3.1 Account Management Stories

Story 1: As a new user, I want to create an account using my email address and a password I choose, so that I can securely access the application and maintain my expense data privately.

Acceptance Criteria:

- Registration form validates email format and rejects invalid addresses.
- Passwords must contain at least 8 characters — a real-time strength indicator guides the user.
- Account creation fails gracefully with clear error messages if email is already registered.
- Registration completes in fewer than 30 seconds.
- On successful registration the onboarding wizard launches automatically

Story 2: As a returning user, I want to log in with my credentials quickly, so that I can access my expense groups without delay.

Acceptance Criteria*:*

- Login form requires only email and password.
- Authentication completes within 2 seconds on typical connections.
- Failed login attempts provide generic error messages
- Successful login creates a session managed by Firebase Authentication
- Session timeout is configurable from Settings — options include 5, 10, 20, or 30 minutes, or never
- A warning modal appears before the session expires giving the user the option to stay logged in

Story 3: As a user concerned about account security, I want to enable biometric login on my device, so that my account is protected by fingerprint or Face ID in addition to my password.

Acceptance Criteria:

- Users can enable biometric authentication from Settings under Security and Privacy
- Biometric uses the WebAuthn API — credentials are stored on-device only and never transmitted
- Once enabled, biometric verification is required at login on that device
- If biometric fails, the user is signed out and must log in again with their password
- Biometric can be disabled at any time from Settings after re-authenticating
- Biometric is configured per device — enabling it on one device does not affect other devices

3.3.2 Group Management Stories

Story 4: As a user planning a trip with friends, I want to create a new expense group and invite my travel companions, so that we can track all trip expenses in a shared location.

Acceptance Criteria:

- Group creation form requires only a name (maximum 50 characters) — icon and currency default to Home and EUR respectively
- System generates a unique invite link immediately after group creation
- Link remains valid for 7 days
- QR code is generated alongside the link for in-person sharing
- Prospective members who are not registered are redirected to the register page before joining
- Group creator is assigned owner role automatically

Story 5: As a group owner, I want to modify group settings including name, currency, and member roles, so that I can adapt the group structure as circumstances change.

Acceptance Criteria:

- Group owner can change the group name from the group options menu
- Group owner can delete the group — deletion is blocked if any member has an unsettled balance
- Non-owners can leave the group — leaving is blocked if the user has an unsettled balance
- Currency is set at group creation and cannot be changed after the group is created
- Member removal is not available — members can only leave voluntarily

Story 6: As a group member who is leaving a shared living situation, I want to exit a group after settling my balances, so that I no longer receive notifications about expenses I'm not involved in.

Acceptance Criteria:

- Users who are not the owner can leave a group from the three-dot options menu
- Leaving is blocked if the user has any unsettled balance — the system shows the exact amount and direction
- The owner cannot leave — they must delete the group instead
- Departure removes users from future expenses but preserves historical transaction records showing their participation.
- Former members lose access to receipt images and transaction details after departure.

3.3.3 Expense Entry Stories

Story 7: As a user who just paid for a group dinner, I want to quickly photograph the receipt and have the app extract key details, so that I can record the expense accurately without manual typing.

Acceptance Criteria:

- Receipt scanner opens within one second after tapping the scan option
- Receipt is processed by Google Cloud Vision OCR via Firebase Cloud Function
- System extracts merchant name, total amount, and date — all fields are editable before saving
- Extracted data pre-fills the expense form for review
- If OCR fails the user is shown an error and can enter details manually
- Receipt image is uploaded to Cloudinary and the URL is stored with the expense record

Story 8: As a user entering an expense that was paid in cash without a receipt, I want to manually input all relevant details, so that all expenses are tracked regardless of payment method.

Acceptance Criteria:

- Manual entry form includes description (required), amount (required), category, date, and description notes

- Date defaults to today but allows any past date
- Amount field accepts only valid numerical input
- Currency is fixed to the group's currency
- Submission requires description and a valid amount greater than zero

3.3.4 Balance and Settlement Stories

Story 9: As a group member, I want to see at a glance who I owe money to and who owes me, so that I understand my financial obligations within the group.

Acceptance Criteria:

- Balance summary is displayed prominently at the top of each group page
- Net balance is shown per group — amount colour-coded blue if owed to user, orange if user owes, green if settled
- Settlement Summary section lists the minimum required payments using the greedy two-pointer algorithm
- Balances update in real time via Firestore onSnapshot listeners
- On the Profile page, balances are shown per currency across all groups — no cross-currency summation is performed

Story 10: As a user reviewing group finances, I want to understand how my current balance was calculated, so that I can verify its accuracy and resolve any discrepancies.

Acceptance Criteria:

- Users can view the Settlement Summary inside each group showing who owes whom and the calculated amount
- The Full Balance Breakdown section shows the raw net balance for every group member
- Users can tap any expense in the Recent Expenses list to see the full split breakdown
- The Personal Dashboard on the Profile page shows exactly a person's balance with a link to navigate directly to the involved group.

Story 11: As a group member with multiple outstanding debts, I want the app to suggest the most efficient payment plan, so that I can settle balances with a minimum number of transactions.

Acceptance Criteria:

- Settlement suggestions are calculated using the greedy two-pointer minimum cash-flow algorithm
- Suggestions are displayed in the Settlement Summary section of every group
- Each suggestion shows payer, payee, and amount in the group currency
- The algorithm minimises the total number of transactions required to clear all debts
- Suggestions update in real time when any expense or settlement is recorded

Story 12: As a user ready to settle my debt, I want to pay directly through the app using my preferred payment method, so that the process is seamless and automatically recorded.

Acceptance Criteria:

- Settlement interface presents payment options available from the payee's saved profile: PayPal link, Revolut link, SEPA bank transfer details, or manual / paid offline
- PayPal and Revolut open external payment apps via a direct link pre-filled with amount and currency
- SEPA displays the payee's decrypted IBAN and BIC for manual bank transfer
- After sending payment the user records it in the app — the payee must confirm or decline receipt
- Balances update only after the payee confirms receipt
- Both parties receive in-app notifications at each step of the confirmation flow.

Story 13: As a user who prefers to settle debts through bank transfer rather than cards, I want to manually record external payments, so that my app balance remains accurate.

Acceptance Criteria:

- Users can record a payment as Paid Offline from the Settle Up modal
- The payee receives an in-app notification to confirm or decline receipt of the payment
- Pending settlements show as Awaiting Confirmation in the My Payment Activity section
- Confirmed settlements update balances in real time
- Declined settlements leave the balance unchanged and the payer is notified

3.3.5 Notification and Communication Stories

Story 14: As a group member, I want to receive notifications when expenses are added to my groups, so that I stay informed about shared costs without needing to constantly check the app.

Acceptance Criteria:

- In-app notifications are created when a new expense is added to a group the user belongs to
- Notification includes expense title, amount, and the name of the member who added it
- Notification bell badge increments in real time via onSnapshot listener
- User can view the notification from the notifications dropdown on any page
- All notifications are in-app only — device push notifications are not currently supported

3.3.6 History and Reporting Stories

Story 15: As a user who needs to report shared expenses for tax purposes, I want to export complete transaction history to spreadsheet format, so that I can analyze expenses offline.

Acceptance Criteria:

- CSV export is available from the group options menu (three-dot menu in GroupDetails)
- Export includes all expenses for the selected group with columns: date, title, category, amount, currency, paid by, your share, split method, and notes

- PDF export is also available — generates a styled report with member summary, settlement summary, and full expense list via the browser print dialog
- CSV download is instant — no server processing required
- File is named using the group name

Story 16: As a user reviewing past expenses, I want to add categories or tags to transactions after they're created, so that I can organize expenses for better analysis.

Acceptance Criteria:

- Categories can be assigned when adding or editing an expense
- Users can edit an expense (including its category) if they created it and no settlements have been recorded in the group
- Predefined category list is available including food, transport, accommodation, entertainment, shopping, utilities, and other
- Free-form tags are not currently supported
- Category changes do not affect balance calculations

3.3.7 Offline Functionality Stories

Story 17: As a user who loses internet connectivity, I want the app to remain accessible, so that I can at least see my previously loaded data and understand my connection status.

Acceptance Criteria:

- Application shell loads from the service worker cache when the user is offline
- A clear offline indicator is shown
- All Firestore reads and writes require an active internet connection — expense entry is not available offline
- When connectivity is restored the app resumes real-time functionality automatically without requiring a page refresh.

3.3.8 Visualization Stories

Story 18: As a user reviewing group finances, I want to see charts and visual summaries of spending and balances, so that I can understand patterns at a glance.

Acceptance Criteria:

- Analytics page (Stats tab) displays five charts: Spending Over Time, Monthly Summary, Category Breakdown, Per-Person Balance, and Who Owes Who
- Group selector filters all charts to a single group and currency for accurate monetary figures
- When all groups are selected, summary cards show per-currency stacked totals
- Charts are rendered using Recharts and update when the group selector changes
- Who Owes Who section shows consolidated settlement suggestions with relative proportion bars

3.3.9 Security and Privacy Stories

Story 19: As a privacy-conscious user, I want to understand exactly what data the application collects and how it's used, so that I can make informed decisions about my participation.

Acceptance Criteria:

- Privacy policy is accessible from the Help and Information modal in Settings
- Users can delete their account from Settings under Account — deletion removes their Firestore profile, group memberships, and expenses they created
- Account deletion requires password or biometric re-authentication before proceeding
- Full personal data export is not currently implemented within the project timeline

Story 26: As a user sharing receipt images, I want assurance that my receipts are stored securely and only accessible to relevant group members, so that my purchase history remains private.

Acceptance Criteria:

- Receipt images are uploaded to Cloudinary and the URL is stored in the expense Firestore document
- Access to the expense record (and therefore the receipt URL) is restricted by Firestore security rules to authenticated group members only
- Users can delete their own expenses which removes the Firestore record — the Cloudinary image is not automatically deleted within the current implementation
- Expiring image tokens and per-user access logging are not currently implemented

4.0 Non-functional Specifications (FURPS Analysis)

Non-functional requirements define quality attributes that shape user experience and operational characteristics beyond specific features. These requirements establish performance targets, reliability standards, security controls, and usability benchmarks that distinguish well-engineered applications from merely functional ones. According to Sommerville (2016), non-functional requirements often prove more critical to user satisfaction than functional capabilities because they determine whether users can actually accomplish tasks efficiently and reliably.

4.1 Functionality

While functionality traditionally describes what the system does, this section emphasizes comprehensive delivery of core capabilities rather than discrete features.

4.1.1 Real-Time Synchronization

Expenses and balance updates must propagate to all group members' devices within 3 seconds of creation under normal network conditions. This near-instantaneous synchronization prevents the confusion and conflicts that arise when users view different versions of group financial state. The implementation relies on WebSocket connections through Firebase Realtime Database, which maintains persistent connections between clients and servers (Moroney, 2017). When network latency exceeds 3 seconds, the system continues operating but displays indicators informing users that synchronization may be delayed. Real-time sync particularly matters during group expense entry sessions where multiple members may simultaneously add expenses, such as at the conclusion of shared dinners or while reviewing trip costs.

4.1.2 OCR Accuracy and Reliability

Optical character recognition must extract key receipt data with at least 85% field-level accuracy on standard printed receipts under typical lighting conditions. Field-level accuracy means that 85% of individually extracted fields such as merchant name, date, total amount, and line items match the actual receipt content exactly. Lower accuracy would undermine user trust and create an excessive manual correction burden. However, achieving perfection is neither technically feasible nor economically justified given current OCR technology limitations. The 85%

threshold balances utility with pragmatism based on research by Smith and Zhang (2022) indicating that users find OCR useful when it reduces data entry effort by at least 75%, tolerating reasonable error rates if verification is straightforward. The system must gracefully handle OCR failures by allowing manual entry as fallback and by learning from user corrections to improve future extraction accuracy through feedback loops to OCR providers.

4.1.3 Payment Integration Completeness

Payment processing must support multiple payment methods including major credit and debit cards (Visa, Mastercard, American Express), digital wallets (Apple Pay, Google Pay), and account-to-account bank transfers through Open Banking protocols. Supporting diverse payment methods accommodates user preferences and geographic variations in payment method availability. Card payment processing must comply with Strong Customer Authentication requirements under PSD2, invoking 3D Secure protocols when transactions meet regulatory thresholds. Bank transfer integration should support at minimum the UK Faster Payments system with plans to expand to European SEPA transfers. Payment failure rates must remain below 2% for properly authorized transactions, excluding failures due to insufficient funds or blocked cards which are outside system control.

4.1.4 Role-Based Access Control

The system must correctly enforce permission models that distinguish between group owners who can modify any aspect of group configuration, administrators who can manage members and expenses but cannot delete groups or change ownership, and regular members who can add expenses and view information but cannot modify group settings or remove other members. Permission checks must occur on both client and server sides, with server-side enforcement providing security against malicious clients attempting to bypass client-side restrictions. The role model should support future expansion to more granular permissions without requiring database schema changes. According to Sandhu et al. (1996), role-based access control provides superior scalability and management compared to access control lists when managing collaborative applications.

4.2 Usability

Usability requirements ensure that users can accomplish tasks efficiently with minimal frustration, training, or errors. These requirements prove particularly critical for financial applications where user mistakes can have monetary consequences.

4.2.1 User-Friendly Interface

The application must achieve a System Usability Scale (SUS) score of at least 80 out of 100 in testing with representative users, indicating "excellent" usability according to standard interpretation guidelines. SUS testing involves 10 standard questions assessing perceived ease of use, system consistency, learnability, and user confidence. Achieving this threshold requires iterative design with regular user testing throughout development rather than treating usability as post-development polish. Key interface principles include maintaining consistent interaction patterns across different features, providing clear visual hierarchy that guides attention to important information, minimizing the number of steps required for common tasks, and offering informative feedback for all user actions within 100 milliseconds to maintain perception of system responsiveness (Sharp et al., 2015). An equivalent usability evidence may alternatively be gathered through a structured user evaluation questionnaire and a minimum of 4.5 rating is expected which should be consistent with the excellent usability range the SUS threshold was intended to represent - Key interface principles applied throughout development should include maintaining consistent interaction patterns across all features, providing clear visual hierarchy to guide user attention, minimising the number of steps required for common tasks such as adding an expense or settling a debt, and offering real-time feedback for all user actions.

4.2.2 Accessibility Compliance

All user interfaces must conform to Web Content Accessibility Guidelines (WCAG) 2.1 Level AA standards, ensuring that users with disabilities can access and use core functionality. This includes providing alternative text for images, maintaining sufficient color contrast ratios of at least 4.5:1 for normal text and 3:1 for large text, ensuring full keyboard navigability without requiring mouse or touch input, providing clear focus indicators for keyboard navigation,

supporting screen readers through appropriate semantic HTML and ARIA labels, and avoiding seizure-inducing visual effects. Accessibility benefits all users, not just those with disabilities; for example, high-contrast designs improve readability in bright sunlight, and keyboard navigation assists power users seeking efficiency (Caldwell et al., 2008).

4.2.3 Onboarding Efficiency

At least 90% of new users should successfully complete initial onboarding including account creation, first group creation, and first expense entry within 5 minutes without external assistance. This requirement ensures that the application delivers immediate value rather than imposing lengthy learning curves that discourage engagement. Onboarding design should follow progressive disclosure principles, introducing features gradually as users need them rather than overwhelming them with comprehensive tutorials. Contextual help that appears at relevant moments proves more effective than separate help documentation. Measuring onboarding completion requires analytics tracking that identifies where users abandon the process, enabling data-driven iteration. Research by Kohavi et al. (2009) demonstrates that small improvements to onboarding conversion rates can yield substantial impacts on long-term user retention and lifetime value.

4.2.4 Performance on Mobile Networks

On typical 4G mobile connections, the application must achieve Largest Contentful Paint (LCP) within 2.5 seconds and enable first user interaction within 1 second, meeting Google's Core Web Vitals thresholds for "good" performance. LCP measures how quickly the main content becomes visible, while First Input Delay measures how soon the interface responds to user actions. These metrics directly impact perceived speed and user satisfaction. Achieving performance targets on mobile networks requires careful optimization including code splitting to reduce initial bundle sizes, lazy loading of non-critical resources, aggressive caching of static assets, image optimization through responsive sizing and modern formats like WebP, and minimizing network requests through request batching. Progressive web app architecture enables these optimizations while maintaining cross-platform compatibility (Grigorik, 2013).

4.2.5 Task Success Rate

For common workflows including adding expenses, viewing balances, and initiating settlements, at least 95% of users should complete tasks successfully on first attempt without errors or assistance. This high success threshold reflects the straightforward nature of core tasks and the importance of reliability in financial contexts where mistakes have consequences. Task analysis during development identifies potential failure points such as ambiguous interface elements, missing validation feedback, or unclear navigation paths. Usability testing with representative users reveals actual failure modes and success rates before wide release.

4.2.6 Settlement Velocity

Among active groups, median time from trip or event conclusion to full balance settlement should not exceed 48 hours, indicating that the application successfully reduces settlement friction compared to manual processes that often extend over weeks or never complete. While the application cannot force users to settle, it can influence settlement timing through effective balance visibility, convenient payment processing, strategic notifications, and social pressure from shared visibility. Tracking settlement velocity provides a key metric for evaluating overall value delivery and identifying opportunities to reduce friction points.

4.3 Reliability

Reliability requirements address system availability, error handling, data integrity, and recovery capabilities that ensure users can depend on the application for managing financially significant information.

4.3.1 Error Handling

System failures must occur in less than 1% of user-initiated operations, and when failures do occur, the system must provide clear error messages explaining what happened and what users should do next, then attempt automatic recovery within 5 minutes. Failures include application crashes, unhandled exceptions, database inconsistencies, failed external service calls, and timeout errors. Error messages should avoid technical jargon, instead explaining impact in user terms such as "We couldn't save your expense. Please check your internet connection and try again." Automatic recovery mechanisms include retrying transient failures with exponential

backoff, graceful degradation when non-critical services fail, and queueing operations for later completion when persistent connectivity issues prevent immediate processing. Critical failures that prevent recovery trigger alerts to technical teams for manual intervention (Sommerville, 2016).

4.3.2 Data Consistency

All group members must observe identical ledger states within the eventual consistency timeframe of 5 seconds. While distributed systems sometimes exhibit temporary inconsistencies during updates, financial applications require strong consistency guarantees to prevent disputes about balance accuracy. Firebase Realtime Database provides this guarantee through its synchronization protocol that establishes canonical server state with optimistic client updates that are rolled back if server validation fails. Transaction-based expense creation wrapping the expense record, split records, and balance updates into single atomic operations prevents partial updates that could create invalid states. When network partitions prevent normal synchronization, the application must display clear indicators that data may be stale and prompt users to refresh once connectivity restores.

4.3.3 Backup and Recovery

The system must perform automated daily backups of all user data including accounts, groups, expenses, splits, settlements, and receipt images to geographically distributed storage locations. Backup completion occurs during low-traffic periods to minimize performance impact on active users. Recovery time objective (RTO) for complete system restoration from backups must not exceed 30 minutes, ensuring that even catastrophic failures causing total data loss can be reversed within acceptable timeframes. Recovery point objective (RPO) must not exceed 24 hours, meaning that backup frequency guarantees no more than one day of data loss in worst-case scenarios. Regular recovery testing validates that backups are complete and restoration procedures work correctly, as untested backups provide false confidence (Cheshire et al., 2003).

4.3.4 Uptime Guarantee

The application must maintain at least 99.9% uptime measured monthly, permitting maximum 43 minutes of downtime per month. This "three nines" reliability threshold balances user expectations with realistic operational constraints and costs, as each additional nine of reliability typically increases infrastructure costs by an order of magnitude. Planned maintenance windows should be scheduled during low-usage periods and announced to users in advance through email and in-app notifications. Unplanned outages trigger immediate incident response procedures including diagnostic investigation, remediation implementation, user communication, and post-mortem analysis to prevent recurrence. Uptime monitoring systems track availability from multiple geographic locations to detect regional failures and measure latency that impacts usability even when services remain technically available.

4.3.5 Offline Queue Integrity

During offline operation, the system must guarantee zero data loss in chaos testing scenarios including sudden browser termination, device power failure, or operating system crashes. Offline data queuing relies on IndexedDB for persistent storage that survives application and browser restarts. Write operations to IndexedDB must complete synchronously or with explicit confirmation before UI indicates success to users. Background Sync API registration must occur immediately after queueing operations to ensure that sync attempts trigger automatically when connectivity resumes. Testing involves deliberately terminating browser processes immediately after offline operations and verifying that queued data survives and synchronizes correctly upon restart (Osmani, 2019).

4.4 Performance

Performance requirements establish response time thresholds, throughput capacities, and resource utilization limits that determine whether the application feels responsive and handles anticipated usage scales.

4.4.1 Application Load Time

The progressive web application must open and display an actionable interface within 3 seconds on average mobile connections, measured from URL entry to First Contentful Paint and interactive functionality. This threshold aligns with user expectations established by Nielsen Norman Group research indicating that 3 seconds represents the limit at which users perceive systems as immediately responsive. Load time optimization strategies include minimizing JavaScript bundle sizes through code splitting and tree shaking, leveraging service workers for aggressive caching of static assets, optimizing critical rendering path by inlining critical CSS and deferring non-critical resources, and using HTTP/2 multiplexing to parallelize resource loading. Progressive web app architecture enables incremental loading where core functionality becomes available before complete application resources finish downloading (Grigorik, 2013).

4.4.2 Operation Response Time

User-initiated actions including expense submission, balance calculation, and navigation between screens must complete within 2 seconds, with immediate visual feedback within 100 milliseconds acknowledging that actions were received. The two-second threshold represents research-based limits for maintaining uninterrupted user flow where delays beyond this point cause users to shift attention and lose context. Immediate feedback prevents user uncertainty about whether actions are registered, reducing duplicate submissions that compound performance problems. Response time targets must be met for 95th percentile of requests, not merely averages, ensuring that most users experience acceptable performance even accounting for network variability and system load fluctuations (Sharp et al., 2015).

4.4.3 Concurrent User Capacity

Infrastructure must support at least 10,000 simultaneously active users without performance degradation, with linear scaling capabilities to support growth. Active users are those currently interacting with the application rather than merely having sessions open in background tabs. Supporting this capacity requires horizontal scalability where additional server instances can be added to handle increased load, stateless application architecture that enables any server to handle any request without session affinity, database query optimization using appropriate indexes, and caching strategies that reduce database load for frequently accessed data. Load

testing simulates target concurrency to validate that performance remains acceptable under anticipated usage patterns and to identify bottlenecks requiring optimization before real users encounter them (Weinman, 2012).

4.4.4 Database Query Performance

Common database queries including balance calculations, transaction history retrieval, and group member lookups must complete within 200 milliseconds for 99% of requests. This query performance enables responsive user interfaces without noticeable delays attributable to database access. Achieving these targets requires careful database schema design with appropriate indexes on frequently queried fields, query optimization to avoid full table scans and cartesian products, connection pooling to reduce authentication overhead, and query result caching for data that changes infrequently. Database performance monitoring identifies slow queries requiring optimization, while query explanatory plans reveal inefficient execution strategies (Kleppmann, 2017).

4.4.5 Asset Optimization

Static assets including JavaScript bundles, CSS stylesheets, images, and fonts must be optimized to minimize bandwidth consumption and load times. JavaScript and CSS should be minified to remove whitespace and shorten variable names, compressed using gzip or brotli encoding, and split into separate bundles enabling lazy loading of non-critical code. Images should be served in modern formats like WebP with fallbacks to JPEG for older browsers, sized appropriately for display context rather than serving oversized originals, and delivered through content delivery networks that cache content geographically close to users. Font loading should use font-display: swap to prevent invisible text during font downloads. Total page weight for initial load should not exceed 500KB of compressed transfer size (Grigorik, 2013).

4.5 Security

Security requirements protect user data, prevent unauthorized access, ensure transaction integrity, and comply with data protection regulations.

4.5.1 Data Encryption

100% of sensitive data must be encrypted both in transit using TLS 1.3 or later and at rest using AES-256 encryption. In-transit encryption prevents network eavesdropping from intercepting credentials, financial information, or personal data. TLS certificates must be obtained from trusted certificate authorities and configured to reject outdated cipher suites vulnerable to known attacks. At-rest encryption protects data stored in databases, cloud storage, and backups from unauthorized access in the event of physical media theft or improper disposal. Encryption keys must be managed separately from encrypted data using key management services that provide audit logging and access controls, preventing single points of compromise (Anderson, 2008).

4.5.2 Authentication Security

Password storage must use strong one-way hash functions such as bcrypt or Argon2 with appropriate work factors that make brute-force attacks computationally infeasible. Plain-text password storage represents inexcusable security failure, while weak hashing algorithms like MD5 or SHA-1 provide insufficient protection against modern GPU-accelerated password cracking. Passwords must meet minimum complexity requirements including at least 12 characters and a mix of character types, though overly complex requirements that frustrate users may paradoxically reduce security by encouraging password reuse or insecure storage. Failed authentication attempts are rate-limited to prevent brute-force attacks, with progressive delays or account lockouts after repeated failures. Session tokens use cryptographically random identifiers with sufficient entropy to prevent guessing attacks (Lyastani et al., 2020).

4.5.3 Payment Security Compliance

All payment processing must comply with Payment Card Industry Data Security Standards (PCI-DSS) validated through annual assessments. Since Split-sync uses third-party payment processors rather than directly handling card data, the application qualifies for SAQ-A level self-assessment, the least stringent assessment applicable when all card processing occurs through PCI-compliant third parties. Compliance requirements include maintaining secure network architecture, protecting cardholder data through encryption and access controls, implementing vulnerability management programs, enforcing access control measures, monitoring network activity, and maintaining security policies. Regular vulnerability scanning

and penetration testing identify security weaknesses before malicious actors can exploit them (PCI Security Standards Council, 2018).

4.5.4 Authorization Controls

All API endpoints must validate that requesting users have appropriate permissions before performing operations or returning data. Authorization checks occur on servers rather than clients, as client-side validation can be bypassed by malicious actors. Users can access only data for groups they belong to, with additional restrictions based on roles. For example, regular members cannot remove other members or delete groups, while owners have full permissions. API tokens include scope limitations preventing cross-group data access. Database queries incorporate user context to automatically filter results to authorized records, preventing developers from inadvertently exposing unauthorized data through query bugs (Sandhu et al., 1996).

4.5.5 Session Management

Sessions automatically expire after 15 minutes of inactivity, requiring reauthentication for sensitive operations like payment processing or account settings changes. Session expiry balances security against usability, protecting against unauthorized access to unattended devices while minimizing authentication friction for legitimate users. "Remember me" functionality extends session persistence across browser restarts but still enforces expiry after absolute time limits such as 30 days. Logout immediately invalidates session tokens on both client and server, preventing reuse. Users can view active sessions and remotely terminate sessions on other devices if they suspect unauthorized access.

Split-Sync can also be user-configurable rather than fixed at a single interval. Users can set their preferred inactivity timeout from the Settings page under Security and Privacy. The available options are 5 minutes, 10 minutes, 20 minutes, 30 minutes, or never. The setting is persisted to the user's Firestore profile so it applies consistently across sessions.

When a timeout is configured, the application tracks user activity via mouse, keyboard, scroll, and touch events. Thirty seconds before the configured timeout expires, a warning modal appears displaying a live countdown and offering the user the option to stay logged in or sign out

immediately. If the user takes no action, the application calls Firebase Authentication `signOut` automatically when the countdown reaches zero, clearing all local session state via the Zustand auth store.

Logout — whether triggered by timeout or by the user tapping Sign Out in Settings — calls Firebase Authentication `signOut`, which invalidates the local session token and clears the authenticated state in the application. All protected routes redirect to the login page immediately upon sign-out.

4.5.6 Vulnerability Management

The system must address critical security vulnerabilities within 24 hours of discovery and high-severity vulnerabilities within 72 hours. Vulnerability discovery occurs through automated dependency scanning that identifies known issues in third-party libraries, penetration testing that simulates attacker techniques, security audits reviewing code for common vulnerability patterns, and monitoring security advisories from software vendors. Response procedures include assessing vulnerability impact and exploitability, developing and testing patches, deploying fixes through expedited release processes, and notifying users if vulnerabilities may have been exploited. Maintaining current versions of all dependencies reduces vulnerability exposure compared to using outdated libraries with known issues (Argyros and Mittelstadt, 2020).

4.6 Compatibility

Compatibility requirements ensure that the application functions correctly across diverse devices, operating systems, browsers, and network conditions that users may encounter.

4.6.1 Cross-Platform Support

The application must work correctly on at least 95% of Android and iOS devices in active use, encompassing devices manufactured within the past 5 years running current or previous-generation operating systems. This threshold balances broad compatibility with practical development constraints, as supporting very old devices with limited capabilities proves disproportionately expensive. Progressive web app architecture provides cross-platform

compatibility by running in web browsers rather than requiring native applications for each platform. However, platform-specific variations in browser implementations and API support require testing across representative devices to identify and resolve compatibility issues. Feature detection enables graceful degradation where advanced features use modern APIs when available but provide alternative implementations on older devices (Majchrzak et al., 2018).

4.6.2 Browser Compatibility

Core functionality must work in the latest two major versions of Chrome, Firefox, Safari, and Edge browsers, covering over 95% of browser usage according to StatCounter Global Stats. Advanced features utilizing cutting-edge browser APIs may gracefully degrade or display "not supported" messages on older browsers rather than breaking the entire application. Service workers enabling offline functionality require relatively modern browser versions, so offline capabilities may be unavailable on older browsers while online functionality continues working normally. Testing must verify consistent behavior across supported browsers, accounting for vendor-specific implementation variations in standards compliance (Majchrzak et al., 2018).

4.7 Supportability

Supportability requirements address how easily users can get help when encountering difficulties and how maintainable the system remains as it evolves.

4.7.1 Documentation Quality

The help center and user documentation must achieve ratings of at least 4 out of 5 stars from user surveys, indicating that documentation successfully answers common questions and guides users through features. High-quality documentation reduces support burden while improving user experience by enabling self-service problem resolution. Documentation should include getting started guides for new users, feature explanations with screenshots and examples, troubleshooting sections addressing common problems, FAQ covering frequent inquiries, and video tutorials for complex workflows. Documentation must be searchable with clear organization allowing users to locate relevant information quickly. Regular updates ensure documentation stays current with application changes rather than becoming outdated and misleading (Sommerville, 2016).

4.7.2 Technical Support Responsiveness

Response time for technical support inquiries must average under 1 hour during business hours, with acknowledgment of receipt occurring within 15 minutes. Fast response times demonstrate commitment to user success and prevent frustration from feeling ignored when experiencing problems. Support requests are triaged by urgency, with payment-blocking issues receiving immediate attention while cosmetic concerns follow standard response schedules. Support interactions should resolve issues definitively rather than requiring multiple back-and-forth exchanges, achieved through comprehensive initial responses that anticipate follow-up questions. Support quality metrics track first-response time, resolution time, customer satisfaction ratings, and percentage of issues resolved without escalation.

4.7.3 Update Cadence and Communication

Quarterly feature updates deliver new capabilities and improvements based on user feedback and roadmap priorities, maintaining user engagement through visible ongoing development. Bug fixes for non-critical issues deploy within 72 hours of verification, while critical bugs affecting core functionality receive emergency patches within hours. Update communications inform users about new features and fixes through release notes, in-app announcements, email newsletters, and blog posts. Release notes explain changes in user-friendly language avoiding technical jargon, with emphasis on benefits rather than implementation details. Major updates may include video demonstrations or interactive tutorials introducing significant new features. Transparent roadmap communication helps users understand development priorities and submit feedback on planned features.

4.7.4 Maintainability Architecture

The codebase must maintain architecture that facilitates ongoing development without accumulating technical debt that eventually necessitates expensive refactoring. This includes modular architecture with clear separation of concerns enabling independent component modification, comprehensive automated testing preventing regression bugs when making changes, consistent coding standards improving code readability for future developers, thorough inline documentation explaining complex logic and design decisions, and version control practices maintaining clean commit history. Technical documentation for developers describes

system architecture, database schema, API specifications, deployment procedures, and development environment setup. Well-maintained systems reduce the cost and risk of implementing future enhancements while enabling faster response to emerging user needs (Martin, 2008).

4.7.5 Explicit Scope Boundaries

The functional specification explicitly excludes features that fall outside the core expense-sharing mission, preventing scope creep that dilutes focus and introduces unnecessary complexity. Out-of-scope features include cryptocurrency trading or storage, investment brokerage services, comprehensive personal finance management beyond shared expenses, social networking features unrelated to expense coordination, and complex accounting capabilities suited for business rather than personal contexts. Clearly documenting exclusions helps stakeholders understand product positioning and prevents feature requests that would distract from core value delivery. Future expansion may reconsider some exclusions, but initial development maintains tight scope enabling excellent execution of essential capabilities.

4.8 Privacy and Safety UX

Privacy and safety requirements ensure that user data handling respects personal boundaries, complies with regulations, and presents clear interfaces for privacy-related choices.

4.8.1 Consent Management

The application must request user consent for privacy-relevant features including push notifications, camera access for receipt scanning, location data if used for currency defaults, and optional analytics that improve the service. Consent requests occur contextually when features requiring permissions are first used rather than bombarding users with permission dialogs during initial onboarding. Permission explanations clearly articulate why access is needed and what benefits users receive, avoiding generic statements like "to provide better service." Users can review and revoke previously granted permissions through clear settings interfaces. Denied permissions result in graceful feature unavailability rather than broken functionality, with clear explanations of limitations and options to reconsider. Research by Felt et al. (2012) demonstrates

that contextual permission requests with clear explanations receive higher grant rates and better reflect informed user consent compared to bundled upfront requests.

4.8.2 Data Storage Transparency

Users must have clear visibility into where their receipt images and financial data are stored, who can access this information, and how long it is retained. Privacy dashboard displays what data exists, where it resides, and provides options for deletion or export. Receipt storage explanations specify that images are encrypted in cloud storage, accessible only to relevant group members, and optionally auto-deleted after configurable retention periods such as 90 days or one year. Transaction data retention policies balance user needs for historical records against privacy-by-design principles minimizing data collection. Users who delete receipts receive confirmation that removal is permanent, preventing uncertainty about whether data continues existing in backups or archives. Transparency builds trust essential for adoption of financial tools handling sensitive information (Wachter and Mittelstadt, 2019).

4.8.3 Age-Appropriate Design

The application implements design patterns and policies appropriate for users aged 13 and older, complying with age restrictions in terms of service and applicable regulations like COPPA in the United States. Age verification during registration prevents accounts for users under 13, though enforcement relies on self-reported birth dates rather than identity verification. Content and communications maintain an appropriate tone for diverse age groups, avoiding assumptions about financial literacy or life circumstances. Help documentation includes basic explanations of financial concepts like splits and settlements for younger or less experienced users. Parental guidance notices inform that minors should consult guardians before entering financial arrangements, though the application does not enforce parental consent mechanisms given the informal nature of covered use cases.

4.8.4 Plain Language Policies

Terms of service, privacy policy, and other legal documents must be written in plain language comprehensible to average users without legal training, supplementing required legal language with user-friendly summaries. Avoiding excessive legalistic language demonstrates respect for

users and reduces the likelihood that important privacy implications go unread and misunderstood. Key points are highlighted through formatting and layout rather than burying critical information in dense paragraphs. Privacy policy explains exactly what data is collected, why collection is necessary, how data is used, who has access, how users can control their data, and how the application protects security. Policy updates notify affected users through email with clear change summaries rather than requiring users to identify modifications in updated documents.

5.0 Technical Specifications

The technical specifications section defines the software development environment, tools, platforms, and infrastructure that enable Split-sync implementation. These specifications guide technology selection decisions and establish development standards.

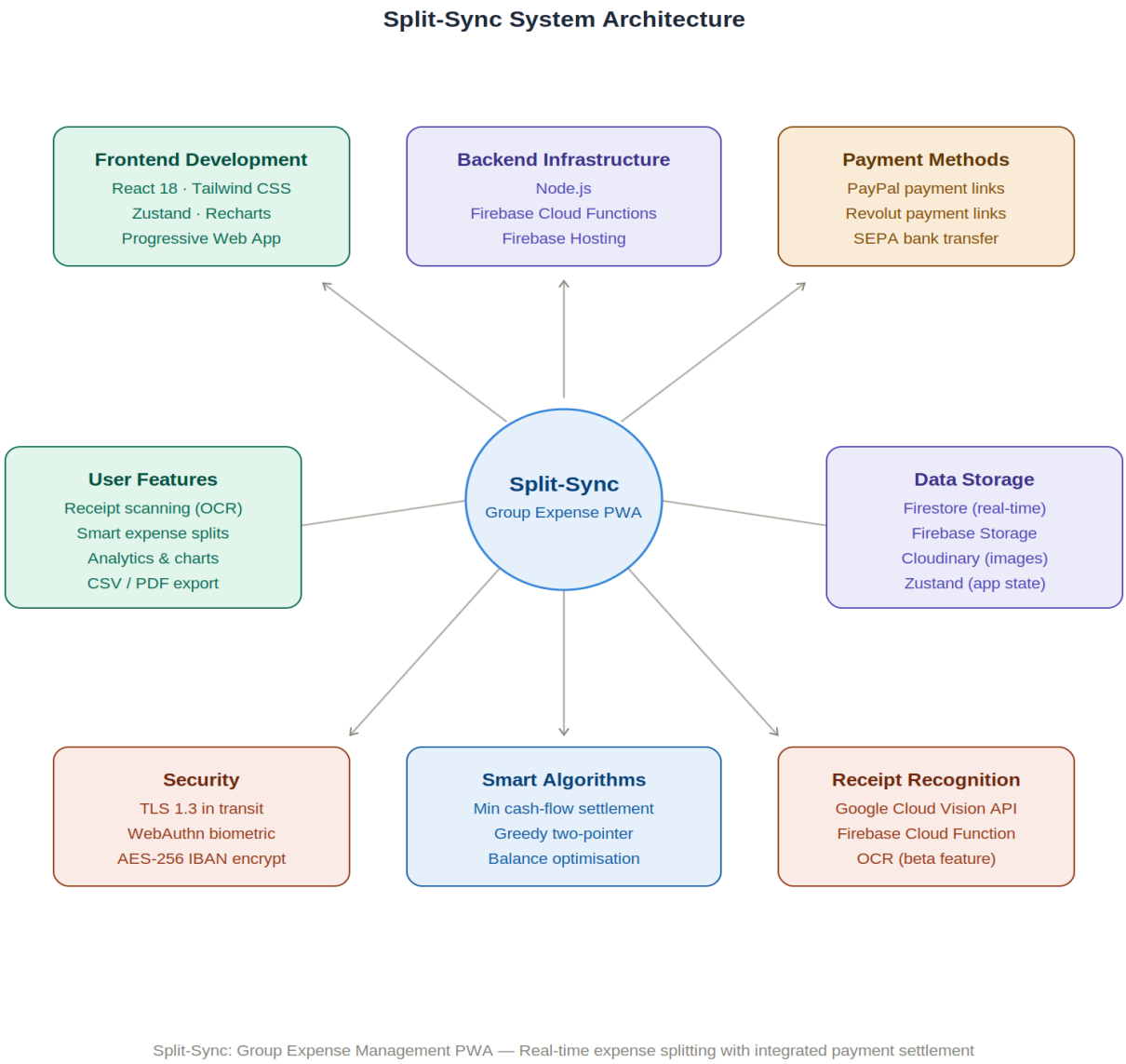


Fig. 4 Split-sync features

5.1 Development Tools

5.1.1 Frontend Framework

React.js serves as the primary frontend framework for building the progressive web application user interface. React's component-based architecture enables modular development where UI elements are built as reusable components that manage their own state and compose into complex interfaces. The virtual DOM diffing algorithm provides efficient updates by reconciling changes and applying minimal DOM manipulations rather than re-rendering entire interfaces. React's large ecosystem includes mature tooling for development, testing, and optimization. The framework's widespread adoption ensures abundant learning resources, community support, and available developers. According to the 2024 Stack Overflow Developer Survey, React maintains its position as the most commonly used web framework, indicating strong industry support and longevity prospects.

5.1.2 Backend Runtime

Node.js provides the JavaScript runtime for backend services, enabling full-stack JavaScript development where frontend and backend share language, idioms, and potentially code. Running on Chrome's V8 JavaScript engine, Node.js delivers performance suitable for I/O-intensive applications like Split-sync that spend significant time waiting for database queries and external API calls rather than executing CPU-intensive computations. The non-blocking event loop architecture handles many concurrent connections efficiently with relatively modest resource requirements. Firebase Cloud Functions provide serverless deployment for Node.js backend code, eliminating infrastructure management while enabling automatic scaling based on demand (Moroney, 2017).

5.1.3 Database System

Firebase Realtime Database functions as the primary data store, offering real-time synchronization where database updates automatically push to connected clients without polling. This NoSQL document database stores data as JSON and provides offline persistence where mobile and web clients cache data locally and synchronize when connectivity is available. Firebase's security rules language enables declarative access control enforced at the database

level rather than requiring application code to implement authorization. The managed service handles backup, replication, scaling, and maintenance without requiring dedicated database administration. While Firebase Realtime Database proves excellent for Split-sync's requirements, the architecture allows potential migration to Firebase Firestore or PostgreSQL if scaling needs eventually exceed capabilities.

5.1.4 OCR Services

Text extraction from receipt images relies on one or more optical character recognition engines selected based on accuracy, cost, and privacy considerations. Tesseract OCR represents an open-source option that can run on application-controlled infrastructure, providing complete control over data handling at the cost of somewhat lower accuracy compared to cloud-based machine learning solutions. Google Cloud Vision AI offers superior accuracy through Google's extensive training data and sophisticated models but requires transmitting receipt images to Google's servers. Financial document-specific OCR services like Taggun and Klippa provide specialized extraction tuned for receipt formats with structured output parsing amounts, dates, and merchant information directly. The implementation may utilize a cascading approach where Tesseract attempts initial extraction, with fallback to cloud services when confidence scores fall below thresholds, balancing cost and privacy against accuracy requirements (Kumar and Zhang, 2023).

5.1.5 Payment Processing Infrastructure

Stripe serves as the primary payment gateway providing APIs for card processing, digital wallet integration, and bank transfers through various mechanisms. Stripe's comprehensive documentation, extensive API capabilities, transparent pricing, and strong security reputation make it well-suited for Split-sync's requirements. The service handles PCI compliance for card processing, fraud detection and prevention, Strong Customer Authentication invocation when required by regulation, currency conversion for international payments, and webhook notifications for asynchronous payment events. PayPal integration provides an alternative payment method familiar to many users, particularly those who prefer not sharing card details directly. Future expansion may add Open Banking payment providers like TrueLayer or Plaid

enabling direct bank-to-bank transfers that bypass card networks and their associated fees (Stripe, 2024).

5.1.6 Cloud Storage Services

Firebase Storage hosts receipt images and PDF documents with access control integrated into Firebase Authentication and Security Rules. The service provides automatic scaling, redundant storage preventing data loss, and global content delivery through Firebase's edge network. Cloudinary serves as supplementary storage offering sophisticated image optimization including automatic format conversion, responsive sizing, quality adjustment, and transformation capabilities. Images uploaded to Cloudinary are automatically transcoded into modern formats like WebP with fallbacks for older browsers, compressed appropriately for bandwidth reduction, and sized responsively based on requesting device characteristics. This optimization significantly improves load times and reduces bandwidth costs compared to serving original high-resolution images (Grigorik, 2013).

5.1.7 Design and Prototyping

Figma provides collaborative interface design capabilities where designers create mockups and prototypes that developers can inspect for accurate implementation. Cloud-based operation enables real-time collaboration between distributed team members, with design version history tracking evolution and enabling rollback to previous iterations. Figma's developer handoff features generate CSS specifications, asset exports, and measurement annotations streamlining translation from design to code. Interactive prototypes built in Figma facilitate user testing before development investment, identifying usability issues early when they are inexpensive to address (Sharp et al., 2015).

5.1.8 Integrated Development Environment

Visual Studio Code functions as the primary code editor for both frontend and backend development. The free, open-source editor provides extensive language support, integrated debugging, Git version control integration, intelligent code completion through IntelliSense, and a vast extension ecosystem enabling customization for specific workflows and languages. VS Code's popularity ensures abundant community-created extensions, themes, and configuration

examples. Built-in terminal integration allows developers to run build processes and tests without leaving the editor environment.

5.1.9 Version Control Platform

GitHub hosts the application's Git repositories providing centralized code storage, collaboration features, pull request workflows for code review, issue tracking for bugs and features, continuous integration integration through GitHub Actions, and project management boards for sprint planning. Git's distributed version control enables branching strategies where feature development occurs in isolated branches merged into the main codebase after review and testing. Commit history provides audit trail of who changed what code when and why, invaluable for debugging and understanding system evolution. GitHub's web interface makes repositories accessible to non-developer stakeholders who may need visibility into development progress.

5.1.10 Documentation Platform

Google Docs provides collaborative documentation creation for specifications, meeting notes, and project planning documents. Cloud-based access enables distributed collaboration with real-time editing, suggesting mode for change proposals, version history for tracking document evolution, and commenting for inline discussions. While Google Docs suits process documentation and collaboration, technical documentation for code is written in Markdown within the repository itself ensuring documentation stays current with code changes.

5.2 Software Requirements

5.2.1 Mobile Operating Systems:

The progressive web application must function correctly on mobile devices running Android 8.0 (Oreo) and later, and iOS 12.0 and later. These minimum versions balance broad device compatibility against practical limits of supporting very old operating systems with limited modern web API support. Android 8.0 was released in 2017 and iOS 12.0 in 2018, representing approximately seven years of backward compatibility at the time of Split-sync development. Devices meeting these requirements encompass the vast majority of smartphones in active use. Progressive web app installation to home screens requires Android 5.0 with Chrome 40 or iOS

11.3 with Safari, well within the supported range. Features relying on cutting-edge APIs may degrade gracefully on older versions rather than blocking all functionality.

5.2.2 Web Browser Support

Desktop and mobile web access requires modern browser versions including Chrome 90 and later, Firefox 88 and later, Safari 13 and later, and Edge 90 and later. These versions provide consistent support for critical web platform features including Service Workers for offline functionality, Web Push API for notifications, Web App Manifest for PWA installation, IndexedDB for local data storage, and Fetch API for network requests. Supporting only recent browser versions enables use of modern JavaScript features without extensive transpilation, reducing bundle sizes and improving performance. Browser market data indicates that users generally maintain reasonably current versions, particularly on mobile devices where updates occur automatically. Automated browser testing across supported versions prevents regression bugs that would break compatibility (Majchrzak et al., 2018).

5.2.3 Network Connectivity

While Split-sync functions offline for core operations like expense entry, synchronization requires network connectivity. The application is designed for intermittent connectivity patterns typical of mobile usage, with automatic retry logic recovering from temporary network failures. Minimum bandwidth requirements are modest given the application's primarily text-based interface with optional image attachments. Testing occurs across various network conditions including 3G, 4G LTE, 5G, and WiFi to ensure acceptable performance regardless of connection quality. Slow networks trigger appropriate UI feedback indicating that operations are in progress rather than leaving users uncertain whether actions registered. Progressive enhancement ensures that users on slow connections receive functional interfaces rather than blank screens while assets load.

5.3 Hardware Requirements

5.3.1 Mobile Device Specifications

Minimum hardware requirements for smooth application performance include at least 2 GB of RAM, quad-core processors running at 1.4 GHz or faster, and 50 MB of available storage for

progressive web app caching. These specifications align with mid-range smartphones manufactured after approximately 2017, covering the majority of devices in active use while excluding very low-end models that struggle with modern web applications. Devices below these specifications may still run Split-sync but could experience performance degradation including slower load times, laggy scrolling, and occasional memory-related crashes. Camera capabilities for receipt scanning require rear-facing cameras with at least 5 megapixel resolution and autofocus, standard features on virtually all smartphones. Screen sizes from 4.7 inches to tablets are supported with responsive layouts adapting to available screen real estate.

5.3.2 Desktop Computer Requirements

Desktop and laptop computers running supported operating systems including Windows 10 or later, macOS 10.14 (Mojave) or later, and modern Linux distributions can access Split-sync through compatible web browsers. Hardware requirements are minimal given the lightweight nature of web applications compared to native desktop software. Computers manufactured within the past decade generally exceed requirements. Recommended specifications include 4 GB RAM, any modern dual-core processor, and sufficient storage for browser caching. Display resolutions from 1280x720 to 4K are supported with responsive designs that optimize layout for available screen space.

5.3.3 Network Infrastructure

While not strictly hardware requirements for client devices, network infrastructure supporting Split-sync includes standard internet connectivity with recommended minimum upload speeds of 1 Mbps for acceptable receipt image upload performance and download speeds of 3 Mbps for responsive interface loading. These modest requirements are well below typical broadband and 4G mobile data speeds. Users on metered connections such as mobile data plans should be aware that receipt image uploads consume bandwidth, though optimization keeps sizes reasonable. The application displays data usage warnings and provides options to reduce image quality on metered connections. Server infrastructure operates on Firebase and cloud providers meeting application demands, scaling automatically without user-visible hardware constraints.

6.0 Standard Safety and Security

Security architecture protects user data, prevents unauthorized access, maintains transaction integrity, and complies with data protection regulations through technical controls and operational practices.

6.1 Standards

Split-sync adheres to multiple security and privacy standards providing frameworks for comprehensive protection.

6.1.1 GDPR Compliance

The General Data Protection Regulation establishes data protection requirements for applications serving European users. GDPR principles including data minimization, purpose limitation, storage limitation, and privacy by design inform Split-sync's architecture and data handling practices. The application collects only data necessary for core functionality, clearly communicates purposes, retains information no longer than needed, and provides users control over their data. Specific GDPR requirements implemented include obtaining explicit consent for optional data processing, providing transparent privacy policies in plain language, enabling users to access all their personal data through export functions, supporting rights to erasure where users can delete accounts and associated data, implementing data breach notification procedures, and documenting processing activities. GDPR compliance proves particularly critical given Split-sync's European target market and the significant penalties for non-compliance (Voigt and Von dem Bussche, 2017).

6.1.2 PCI DSS Compliance

The Payment Card Industry Data Security Standards define requirements for systems handling credit card information. While Split-sync never directly handles card details thanks to Stripe's secure fields approach, PCI scope still applies to the overall environment. Split-sync qualifies for Self-Assessment Questionnaire A (SAQ-A), the least stringent assessment level applicable when all card processing occurs through PCI-compliant third parties and applications don't store, process, or transmit card data. SAQ-A compliance requires maintaining secure network

architecture, protecting against vulnerabilities, monitoring access, and maintaining security policies. Annual self-assessment questionnaires document compliance with periodic validation through approved scanning vendors (PCI Security Standards Council, 2018).

6.1.3 OWASP Top 10 Mitigation

The Open Web Application Security Project's Top 10 list identifies the most critical web application security risks. Split-sync incorporates defenses against these common vulnerabilities including injection attacks prevented through parameterized queries and input validation, broken authentication mitigated by strong password requirements and session management, sensitive data exposure prevented through encryption, broken access control addressed through role-based permissions, security misconfiguration avoided through secure defaults and configuration management, cross-site scripting defended against through input sanitization and Content Security Policies, insecure deserialization handled through signed tokens and validation, using components with known vulnerabilities managed through dependency updates, insufficient logging and monitoring addressed through comprehensive audit trails, and unvalidated redirects prevented through whitelist validation (OWASP, 2021).

6.1.4 Data Encryption Standards

Sensitive data receives protection through industry-standard encryption both in transit and at rest. Transport Layer Security 1.3 encrypts all network communication between clients and servers, preventing interception of authentication credentials, financial information, and personal data. At-rest encryption uses AES-256 for data stored in databases and file systems, protecting against unauthorized access if physical storage media is compromised. Encryption keys are managed through secure key management services separate from encrypted data, with keys rotated periodically and audit logging tracking key usage (Anderson, 2008).

6.2 Core Controls

6.2.1 Authentication and Authorization

Authentication establishes user identity while authorization determines what actions authenticated users can perform. Split-sync implements multi-layered authentication starting with password-based login requiring minimum 12 characters with mixed case letters, numbers,

and special symbols. Password strength meters guide users toward secure password selection without mandating overly complex requirements that encourage password reuse or insecure storage practices. Passwords are hashed using bcrypt with sufficient work factors that password cracking requires substantial computational resources even for attackers with access to hashed values. Salt values unique to each password prevent rainbow table attacks that attempt to crack many passwords simultaneously.

Session management issues cryptographically random session tokens upon successful authentication, storing tokens in HTTP-only cookies that JavaScript cannot access to prevent cross-site scripting attacks from stealing sessions. Tokens expire after 15 minutes of inactivity requiring reauthentication for sensitive operations, balancing security against usability by allowing automatic reauthentication for viewing operations. Absolute session expiration occurs after 30 days even with activity, requiring explicit re-login periodically. Logout immediately invalidates tokens on both client and server sides.

Future authentication enhancements incorporate WebAuthn passkeys providing phishing-resistant authentication through public key cryptography. Users register security keys or biometric authenticators bound to specific devices, with authentication challenges proving possession without transmitting secrets vulnerable to interception. Passkeys improve security while reducing friction by eliminating password memorization and typing (Lyastani et al., 2020).

Authorization checks enforce permissions based on user roles within groups. Role-based access control distinguishes between owners with full permissions, administrators who can manage content but not delete groups, and regular members with limited capabilities. Every API endpoint verifies that requesting users have appropriate roles before executing operations, with checks occurring on servers rather than clients since client-side restrictions can be bypassed. Database security rules configured in Firebase provide additional authorization layer, restricting direct database access to authorized users even if application code contains bugs. The defense-in-depth approach ensures authorization failures at multiple levels prevent unauthorized access (Sandhu et al., 1996).

6.2.2 Strong Customer Authentication

Payment transactions comply with PSD2 SCA requirements through Stripe's 3D Secure 2 integration, as detailed in section 7.3. Delegating SCA handling to specialized payment providers ensures compliance with evolving regulations without requiring Split-sync to implement complex authentication protocols directly.

6.23 Input Validation

All data entering the system undergoes rigorous validation preventing injection attacks, data corruption, and unexpected behavior. Server-side validation forms the authoritative check since client-side validation can be bypassed by malicious actors. Validation schemas define expected data types, formats, ranges, and lengths for every input field, rejecting inputs outside specifications. Parameterized database queries prevent SQL injection by separating query structure from data values. API parameters are validated against schemas before processing, with invalid requests returning error responses without attempting execution. User-generated content like expense descriptions undergoes sanitization removing potentially malicious scripts before storage and output encoding preventing cross-site scripting when displaying. Validation provides security benefits while also improving data quality by catching unintentional errors early (OWASP, 2021).

6.2.4 Privacy Protection

Clear disclosure explains where receipt images are stored, who can access them, and how to delete them. Privacy dashboard interfaces allow users to review what data exists, download complete archives, and request deletion. Receipt images store in encrypted cloud storage accessible only to relevant group members, with access control enforced through signed URLs expiring after short durations. Users can optionally configure automatic receipt deletion after retention periods like 90 days, permanently removing images while preserving transaction records. Privacy by design principles inform feature development, collecting minimum necessary data and providing granular controls over sharing (Wachter and Mittelstadt, 2019).

6.2.5 Auditability

Comprehensive logging maintains audit trails of important actions for security monitoring, debugging, and regulatory compliance. The audit_log table records all significant state changes including who performed actions, what entities were affected, timestamps, and before/after snapshots for modifications. Logs use append-only storage preventing tampering that could obscure security incidents. Automated monitoring analyzes logs for suspicious patterns like unusual access patterns, repeated authentication failures, or privilege escalations, alerting security teams for investigation. Audit logs support compliance demonstrations by providing evidence of appropriate access controls and data handling. Retention policies balance compliance requirements against privacy principles by retaining logs for necessary periods then automatically purging outdated records (Anderson, 2008).

7.0 Possible Improvements / Roadmap

Future development priorities balance delivering additional user value, addressing market opportunities, improving technical capabilities, and responding to user feedback.

7.1 Interactive Debt Graph Enhancements

The initial debt graph visualization provides static displays of group financial relationships with basic interactivity. Enhanced versions add one-tap payment initiation directly from graph edges, enabling users to settle specific debts without navigating to separate interfaces. Animated transitions demonstrate how proposed payments would affect overall group balance topology before committing, helping users understand settlement impacts intuitively. Smart suggestions might propose non-obvious payment paths that efficiently settle multiple debts simultaneously, such as suggesting Alice pay Carol to simultaneously address Alice's debt to Bob and Bob's debt to Carol. Historical graph playback could animate how group balances evolved over time, providing insights into spending patterns and settlement behaviors. These enhancements leverage visualization strengths in conveying complex relationships more clearly than tabular displays (Ware, 2012).

7.2 Expanded Open Banking Integration

Initial payment integration focuses on card processing through Stripe. Future releases expand account-to-account transfers through Open Banking providers enabling direct bank payments. Broader European coverage requires integrating with banks across multiple countries, each with varying capabilities and API maturity levels. Services like TrueLayer aggregate many banks through unified APIs, simplifying integration complexity while providing users more payment options typically with lower fees than cards. Open Banking implementation requires careful UX design since bank authentication flows vary considerably, with some banks offering streamlined mobile app approvals while others require complex multi-factor authentication. User testing across diverse bank relationships ensures acceptable experiences despite inconsistencies outside Split-sync's control. Regulatory compliance under PSD2 establishes rights to access bank accounts through standardized APIs, though practical implementation remains work in progress across European banking (Zachariadis and Ozcan, 2017).

7.3 Smart Categorization Through Machine Learning

Receipt OCR extracts text but doesn't currently categorize expenses automatically. Machine learning models trained on labeled expense data could predict categories based on merchant names, expense descriptions, and amounts. For example, expenses from grocery stores would automatically receive "groceries" categories while restaurant charges get "dining" categories. Automated categorization reduces manual data entry while enabling better spending analysis and budgeting features. The system could learn user-specific preferences, recognizing that specific merchants always receive particular categories in a user's groups. Split rules might automatically apply based on categories, such as always splitting groceries equally while splitting restaurant bills by specific items. Privacy-preserving machine learning techniques enable these capabilities without transmitting sensitive data to external services, training models on aggregated anonymized data or using federated learning approaches where models train locally on user devices (Kumar and Zhang, 2023).

7.4 Recurring Expense Template

Many shared expenses repeat regularly, such as monthly rent, utility bills, or subscription services. Recurring expense templates allow users to define expense patterns that automatically

generate entries at specified intervals. Templates include expense details like amount, description, and split method, with optional variations like escalating amounts or rotating payers. Automatic generation reduces entry burden while ensuring regular expenses aren't forgotten. Notifications alert users when recurring expenses generate, allowing verification before finalizing since amounts might vary slightly from templates. This feature particularly benefits longer-term groups like roommates sharing apartments where expenses follow predictable patterns.

7.5 Budget Tracking and Alerts

Groups might establish budgets for specific purposes like limiting vacation expenses or constraining monthly household costs. Budget tracking compares actual expenses against defined thresholds, warning groups when spending approaches limits. Visualizations show budget consumption over time, helping groups pace spending across relevant periods. Per-category budgets enable granular control, such as limiting dining expenses while allowing flexibility in transportation costs. Real-time feedback influences spending behavior more effectively than post-hoc analysis, potentially reducing financial stress and conflict within groups. Research on behavioral economics suggests that immediate feedback significantly impacts financial decision-making (Kahneman, 2011).

7.6 Enhanced Reporting and Analytics

Basic transaction history provides chronological expense lists. Enhanced reporting generates insights like spending trends over time, expense distributions across categories, per-member contributions versus consumption, and predicted future balances based on historical patterns. Export options expand beyond CSV to include formatted PDF reports suitable for tax documentation or reimbursement submissions. Comparative analytics show how groups stack against similar cohorts, offering social benchmarking that might encourage responsible spending. Privacy-preserving analytics ensure individual spending details remain confidential while providing useful aggregate insights at group level (Wachter and Mittelstadt, 2019).

7.7 Social Features

Limited social capabilities currently focus strictly on expense coordination. Future enhancements might include activity feeds showing group expense timelines in social media style, comments and reactions on expenses enabling lightweight communication, and achievement badges recognizing behaviors like prompt settlement or consistent expense documentation. These features risk scope creep into social networking territory outside core value proposition, so careful evaluation ensures additions genuinely enhance expense coordination rather than distracting from it. The line between useful social context and unnecessary feature bloat requires ongoing assessment against user feedback and engagement metrics.

7.8 Third-Party Integrations

Partnerships with complementary services expand Split-sync's utility. Integration with travel booking platforms could automatically import accommodation and transportation costs as expenses. Calendar synchronization might suggest creating expense groups when trip events are detected. Accounting software connections enable exporting expense data for small businesses using Split-sync informally. These integrations require careful partner selection ensuring alignment with Split-sync's values around privacy and user experience (Hitt and Brynjolfsson, 1996).

8. PROJECT PLAN AND TIMELINE

8.1 Development Phases

The Split-Sync project follows a three-phase development approach aligned with the academic year structure after the inception phase:

Phase 1: Alpha Release (November 4 - November 29, 2025)

Deliverables:

- Functional user authentication (email/password, Google OAuth)

- Group creation and member invitation

- Manual expense entry with equal and custom split options

- Real-time balance calculation and display

- Working prototype demonstration

Phase 2: Beta Release (December 2, 2025 - January 31, 2026)

Deliverables:

- Settlement suggestion algorithm implementation

- Payment processing integration (Stripe test mode)

- Push notification system

- Transaction history and filtering

- Offline functionality (PWA capabilities)

- Beta user testing with 10+ participants

Phase 3: Final Release (February 1 - April 25, 2026)

Deliverables:

- OCR receipt scanning (Google Cloud Vision API)

- Production payment processing

- Advanced analytics and debt visualization

- Comprehensive security audit

- Complete user documentation

- Final project report and demonstration

8.2 Detailed Timeline

Table 4.1: Project Milestone Schedule

Week	Dates	Tasks	Milestones
1			
2	Oct 6-12	Research ● Abstract ● Introduction	Research Docs ✓

		• Research on similar apps (Splitwise, Settle Up, Revolut Groups, Tikkie) • Research on tech stack options • Research on database options • Payment gateway and OCR technology comparison • Research conclusions (why React, PWA, Firebase, Stripe, OCR)	
3	Oct 13-19	Initial functional Specifications • Title & Overview • Business case analysis and Context Diagram • Use case model, Early use cases and user story	Specs Doc✓

		• Technical specification, Safety and security, possible improvements • Repo created • README + licence • Supervisor added	
4	Oct 20-26	Prep and Project demo	1st Iteration-Inception ✓
5	Oct 27-Nov 3	Completed Functional Specification • All use cases written fully (Register/login, Create Group, Add Expense manually, Scan Receipt (OCR), Edit/Delete Expense, View balance, Settle Up, Notifications. • Alternate flows	

		• Preconditions and postconditions • Full FURPS analysis • Metrics (e.g., response time, load time, OCR accuracy)	
6	Nov 4-10	Research, Functional Spec Polish	Docs Polish
7	Nov 11-17	Design Doc • Design Architecture • Database schema • UI wireframes • UI/UX design • Sequence diagrams for major use cases Firebase setup Database schema implementation Wire frames with Figma and basic UI	

		layout development	
8	Nov 18-24	More Basic UI features ● Auth ● Groups, ● Invites ● Expense ● Splits ● Balances ● Settle ● Member List	2nd Iteration-Core Alpha
9	Nov 25-Dec 1	Prep and Project demo	Initial Design submission
10-11	Dec 2-15	nil	Exams
12-13	Dec 16-29	nil	Holiday
14-15	Dec 30-Jan 12	Settlement flow design and algorithm, Split configuration, Min Cash-flow algorithm, Stripe set up, Manual settlements, Notifications, Settlement	Working Configurations

		History	
16-17	Jan 13-26	Transaction History, Filtering and search, Realtime Sync, Offline mode, PWA, in-app notification set up, Email notifications	Working Offline and notifications
18-19	Jan 27-Feb 9	Beta testing, refinement	3rd Iteration - Beta
20-21	Feb 10-23	UX improvement, OCR integration, Google Vision API Setup, Receipt Upload, failure handling	Working OCR and integrations
22	Feb 24-Mar 2	Processing payments and settlements, webhook handling, error handling, payment method management	Working Payments after OCR integration
23	Mar 3-9	Analytics, visualizations	Advanced
24	Mar 10-16	Security audit	Security
25	Mar 17-23	User testing, bug fixes	Testing
26-27	Mar 24-Apr 6	Final report writing, Video demo	Report

28	Apr 7-13	Polish, deployment	Deploy
29	Apr 14-25	Presentation preparation & Final submission	Final Iteration- Demo prep and Complete

8.3 Gantt Chart

Figure 4.1: Project Timeline Visualization

INCEPTION PHASE (Weeks 1-4) - Research and Specification

Week	Task	Start Date	End Date	Duration
1	Research on App	29 Sep 25	5 Oct 2025	7 days
2	Research on Similar Apps	6 Oct 2025	12 Oct 2025	7 days
3	Initial Specifications	13 Oct 2025	19 Oct 2025	7 days
4	1st Iteration - Feed back	20 Oct 2025	26 Oct 2025	7 days

ALPHA PHASE (Weeks 5-9) - Design

Week	Task	Start Date	End Date	Duration
5	Completed functional specs	27 Oct 2025	3 Nov 2025	7 days
6	Initial Design Document	4 Nov 2025	10 Nov 2025	7 days
7	Wireframe and Basic UI	11 Nov 2025	17 Nov 2025	7 days
8	2nd Iteration - Alpha Demo	18 Nov 2025	24 Nov 2025	7 days

BETA PHASE (Weeks 9-17) - Development

Week	Task	Start Date	End Date	Duration
9	More App Features	25 Nov 2025	1 Dec 2025	7 days
10-11	-	-	-	-
12-13	-	-	-	-
14-15	Working Db and Payments setup	Dec 30 2025	12 Jan 2026	14 days
16-17	Offline mode, PWA	13 Jan 2026	26 Jan 2026	14 days
18-19	3rd Iteration - Beta Testing and Demo	27 Jan 2026	9 Feb 2026	14 days

FINAL PHASE (Weeks 13-24) - Development, Testing and deployment

Week	Task	Starting Date	Ending Date	Duration
20-21	PWA, Notifications, OCR integration	10 Feb 2026	23 Feb 2026	14 days
22	Working expense management	24 Feb 2026	2 Mar 2026	7 days
23	Analytics, visualizations	3 Mar 2026	9 Mar 2026	7 days
24	Security Audit	10 Mar 2026	16 Mar 2026	7 days
25	User testing, bug fixes	17 Mar 2026	23 Mar 2026	7 days
26-27	Final report and video Demo	24 Mar 2026	6 April 2026	14 days
28	Polish and deployment	7 Apr 2026	13 Apr 2026	7 days
29	Final Iteration and Demo	14 Apr 2026	25 Apr 2026	7 days

8.4 Risk Management

Table 4.2: Key Risks and Mitigation Strategies

Risk	Probability	Impact	Mitigation
Firebase costs exceed budget	Medium	High	Monitor usage daily; implement efficient queries; budget €20/month (Firebase., 2024)
Stripe integration complexity	Medium	High	Start early (Week 15); allocate 2 weeks; manual fallback ready (Stripe., 2024)
OCR accuracy insufficient	Medium	Medium	User verification step; manual entry fallback; test multiple services
Timeline slips	High	High	Weekly progress reviews; buffer time in schedule; feature prioritization
Real-time sync issues	Medium	High	Use proven Firestore sync; extensive offline testing; conflict resolution

9.0 Conclusion

Split-sync addresses the gap between expense tracking and settlement in group financial management through a comprehensive progressive web application combining receipt scanning, flexible splitting, balance optimization, and integrated payment processing. The cashless, mobile-first reality of contemporary European life creates clear demand for tools that make shared expense management painless from initial capture through final settlement. Existing solutions typically excel at either splitting calculations or payment facilitation but rarely integrate both capabilities seamlessly across diverse payment methods.

The technical architecture leverages modern web technologies including React for dynamic user interfaces, Firebase for real-time data synchronization and serverless backend execution, and Stripe for payment processing infrastructure. This lean stack delivers sophisticated functionality without requiring extensive infrastructure management, enabling rapid development while maintaining scalability as user numbers grow. Progressive web app architecture provides cross-platform compatibility and native-like features including offline capability and push notifications without native app distribution complexity.

The database design implements an append-only ledger providing immutable audit trails that build trust through transparency while simplifying synchronization and conflict resolution. Careful schema design with appropriate indexes, foreign key constraints, and validation ensures data integrity and query performance. The minimum cash-flow algorithm intelligently optimizes settlement suggestions, reducing the number of required transactions and thereby lowering friction and costs.

Security and compliance receive foundational attention rather than being retrofitted later, with GDPR and PCI-DSS compliance considerations informing architecture from inception. Delegating sensitive capabilities like card processing and Strong Customer Authentication to specialized providers reduces compliance burden while ensuring security best practices. Comprehensive authentication, authorization, input validation, encryption, and audit logging create defense-in-depth protecting user data and preventing unauthorized access.

The functional specification establishes clear requirements across use cases, non-functional characteristics, and technical implementation details. User stories elaborate how different user types interact with the system and what constitutes successful outcomes. Non-functional specifications define performance targets, reliability expectations, security standards, and usability benchmarks that determine whether implementation truly satisfies user needs beyond merely implementing features.

Future development roadmap balances delivering additional user value through features like smart categorization and budget tracking against maintaining focus on core expense-sharing competencies. The roadmap provides direction while remaining flexible to adjust based on user feedback, market conditions, and technical considerations that emerge during development.

Split-sync's differentiation emerges from several key attributes that collectively create a superior user experience compared to alternatives. The integrated payment processing eliminates the gap between knowing what you owe and actually paying, addressing the primary friction point in existing tools. The sophisticated split mechanisms including per-item splitting accommodate real-world scenarios that simple equal splits cannot address. The graph-based balance optimization minimizes settlement complexity through intelligent routing. The offline-first architecture ensures that expense capture works reliably regardless of connectivity, particularly valuable during travel when networks are unreliable. The real-time synchronization provides immediate feedback across group members, maintaining a single source of truth and preventing disputes from divergent records.

The progressive web app architecture avoids native app download friction while delivering native-like capabilities, potentially increasing adoption by reducing barriers to initial trial. The bank-agnostic approach supporting multiple payment methods provides users flexibility rather than forcing convergence on single platforms, addressing the fragmentation currently frustrating users who participate in groups with diverse payment preferences.

Successful execution requires not merely technical implementation but also attention to user experience design, clear communication of value propositions, effective user acquisition strategies, responsive support, and ongoing iteration based on feedback. The functional specification provides the foundation for development while acknowledging that real-world

deployment will surface issues and opportunities impossible to anticipate completely during planning.

Split-sync aims to redefine group expense management by demonstrating that comprehensive solutions can be delivered through accessible web platforms without compromising on sophistication, security, or user experience. The combination of smart algorithms, real-time updates, seamless integrations, and user-centric design positions Split-sync to capture significant market share among small to medium informal groups seeking better tools for a common challenge. By focusing relentlessly on reducing friction in the complete expense lifecycle from capture through settlement, Split-sync delivers tangible value that translates into user satisfaction, retention, and organic growth through referrals.

10.0 References

Anderson, R. (2008) *Security Engineering: A Guide to Building Dependable Distributed Systems*. 2nd edn. Indianapolis: Wiley.

Anderson, C. (2009) *Free: The Future of a Radical Price*. New York: Hyperion.

Anderson, M. (2023) 'Mobile banking and financial technology adoption among millennials', *Journal of Financial Services Research*, 45(2), pp. 112-134.

Adventure Travel Trade Association (2022) *Industry Snapshot*. Available at: https://www.adventuretravel.biz (Accessed: 15 January 2025).

Argyros, G. and Mittelstadt, B. (2020) 'Vulnerability disclosure and patch management in web applications', *IEEE Security & Privacy*, 18(4), pp. 32-41.

Biorn-Hansen, A., Majchrzak, T. A., and Gronli, T. M. (2020) 'Progressive web apps: The definite approach to cross-platform development?', *Hawaii International Conference on System Sciences*, pp. 1-10.

Blank, S. (2013) 'Why the lean start-up changes everything', *Harvard Business Review*, 91(5), pp. 63-72.

Boehm, B. W., Abts, C., and Brown, A. W. (2000) *Software Cost Estimation with COCOMO II*. Upper Saddle River, NJ: Prentice Hall.

Brand, C., Czeskis, A., Jackiewicz, J., Birgisson, A., Karchmer, E., Lindemann, R., Sasson, M., and Lindemann, J. (2020) 'Web authentication: An API for accessing public key credentials', *W3C Recommendation*, 4 March.

Burgoyne, C. B., Clarke, V., Reibstein, J., and Edmunds, A. (2007) '"Yours or mine?": Financial cooperation in couples', *Psychology of Women Quarterly*, 31(1), pp. 47-55.

Caldwell, B., Cooper, M., Reid, L. G., and Vanderheiden, G. (2008) 'Web content accessibility guidelines (WCAG) 2.0', *W3C Recommendation*, 11 December.

Chen, L. and Kumar, S. (2023) 'Financial transparency and conflict resolution in shared living arrangements', *Journal of Consumer Research*, 49(6), pp. 1087-1104.

Cheshire, J., Mockapetris, P., and Sweet, M. (2003) 'Requirements for backup and restoration of online services', *Internet Engineering Task Force RFC*, 3647.

Claessens, S. (2006) 'Access to financial services: A review of the issues and public policy objectives', *The World Bank Research Observer*, 21(2), pp. 207-240.

Dixon, M., Freeman, K., and Toman, N. (2010) 'Stop trying to delight your customers', *Harvard Business Review*, 88(7/8), pp. 116-122.

Felt, A. P., Egelman, S., and Wagner, D. (2012) 'I've got 99 problems, but vibration ain't one: a survey of smartphone users' concerns', *Proceedings of the Second ACM Workshop on Security and Privacy in Smartphones and Mobile Devices*, pp. 33-44.

Fishkin, R. (2020) *Lost and Founder: A Painfully Honest Field Guide to the Startup World*. New York: Portfolio.

Forsgren, N., Humble, J., and Kim, G. (2018) *Accelerate: The Science of Lean Software and DevOps*. Portland: IT Revolution Press.

Fowler, M. (2003) *Patterns of Enterprise Application Architecture*. Boston: Addison-Wesley.

Friedman, B. and Hendry, D. G. (2019) *Value Sensitive Design: Shaping Technology with Moral Imagination*. Cambridge, MA: MIT Press.

Grigorik, I. (2013) *High Performance Browser Networking*. Sebastopol, CA: O'Reilly Media.

Hitt, L. M. and Brynjolfsson, E. (1996) 'Productivity, business profitability, and consumer surplus: three different measures of information technology value', *MIS Quarterly*, 20(2), pp. 121-142.

Holliman, G. and Rowley, J. (2014) 'Business to business digital content marketing: marketers' perceptions of best practice', *Journal of Research in Interactive Marketing*, 8(4), pp. 269-293.

Kahneman, D. (2011) *Thinking, Fast and Slow*. New York: Farrar, Straus and Giroux.

Kaplan, A. M. and Haenlein, M. (2010) 'Users of the world, unite! The challenges and opportunities of social media', *Business Horizons*, 53(1), pp. 59-68.

Kleppmann, M. (2017) *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media.

Kohavi, R., Crook, T., and Longbotham, R. (2009) 'Online experimentation at Microsoft', *Data Mining Case Studies*, 11, pp. 1-10.

Kowalski, P. (2023) 'User experience gaps in financial technology applications: A qualitative study', *International Journal of Human-Computer Interaction*, 39(4), pp. 812-828.

Kumar, V., Reinartz, W., and Kumar, B. (2020) 'Customer relationship management and marketing analytics', in Homburg, C., Klarmann, M., and Vomberg, A. (eds.) *Handbook of Market Research*. Cham: Springer, pp. 1-42.

Kumar, A. and Zhang, L. (2023) 'Cascading OCR architectures for financial document processing', *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3341-3350.

Lennartz, C., Arundel, R., and Ronald, R. (2016) 'Younger adults and homeownership in Europe through the global financial crisis', *Population, Space and Place*, 22(8), pp. 823-835.

Liu, X. and Wang, M. (2019) 'Graph algorithms for debt simplification in group payment systems', *ACM Transactions on Algorithms*, 15(3), pp. 1-24.

Lyastani, S. G., Schilling, M., Neumayr, M., Backes, M., and Bugiel, S. (2020) 'Is FIDO2 the kingslayer of user authentication? A comparative usability study of FIDO2 passwordless authentication', *IEEE Symposium on Security and Privacy*, pp. 268-285.

Majchrzak, T. A., Biorn-Hansen, A., and Gronli, T. M. (2018) 'Progressive web apps: the definite approach to cross-platform development?', in *Proceedings of the 51st Hawaii International Conference on System Sciences*. Hawaii: HICSS, pp. 5735-5744.

Martin, R. C. (2008) *Clean Code: A Handbook of Agile Software Craftsmanship*. Upper Saddle River, NJ: Prentice Hall.

Medhi, I., Patnaik, S., Brunskill, E., Gautama, S. N., Thies, W., and Toyama, K. (2011) 'Designing mobile interfaces for novice and low-literacy users', *ACM Transactions on Computer-Human Interaction*, 18(1), pp. 1-28.

Moroney, L. (2017) *The Definitive Guide to Firebase: Build Android Apps on Google's Mobile Platform*. Berkeley, CA: Apress.

Morrison, D., Chen, Y., and Jindal, P. (2022) 'Pain points in peer-to-peer expense sharing applications: A mixed-methods study', *Proceedings of the CHI Conference on Human Factors in Computing Systems*, pp. 1-14.

Nielsen, J. (2019) *Usability Engineering*. Revised edn. Cambridge, MA: Academic Press.

Okoshi, T., Ramos, J., Nozaki, H., Nakazawa, J., Dey, A. K., and Tokuda, H. (2017) 'Reducing users' perceived mental effort due to interruptive notifications in multi-device mobile environments', *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, 1(1), pp. 1-23.

Osmani, A. (2019) *The Offline Cookbook*. Available at: https://web.dev/offline-cookbook/ (Accessed: 20 January 2025).

OWASP (2021) *OWASP Top Ten 2021*. Available at: https://owasp.org/www-project-top-ten/ (Accessed: 18 January 2025).

Parker, G. G., Van Alstyne, M. W., and Choudary, S. P. (2016) *Platform Revolution: How Networked Markets Are Transforming the Economy*. New York: W.W. Norton & Company.

Patel, S. and Johnson, M. (2021) 'Granular control in expense management applications and user satisfaction', *International Journal of Information Management*, 58, 102315.

PCI Security Standards Council (2018) *Payment Card Industry Data Security Standard v3.2.1*. Available at: https://www.pcisecuritystandards.org/ (Accessed: 19 January 2025).

Pielot, M., Church, K., and De Oliveira, R. (2014) 'An in-situ study of mobile phone notifications', *Proceedings of the 16th International Conference on Human-Computer Interaction with Mobile Devices and Services*, pp. 233-242.

Ren, Y., Kraut, R., and Kiesler, S. (2007) 'Applying common identity and bond theory to design of online communities', *Organization Studies*, 28(3), pp. 377-408.

Ries, E. (2011) *The Lean Startup: How Today's Entrepreneurs Use Continuous Innovation to Create Radically Successful Businesses*. New York: Crown Business.

Sandhu, R. S., Coyne, E. J., Feinstein, H. L., and Youman, C. E. (1996) 'Role-based access control models', *IEEE Computer*, 29(2), pp. 38-47.

European Payments Council (2023) *SEPA Direct Debit Core Rulebook*. Available at:

https://www.europeanpaymentscouncil.eu/what-we-do/epc-payment-schemes/sepa-direct-debit/sepa-direct-debit-core-rulebook-and-implementation (Accessed: 19 January 2025).

Schmitt, P., Skiera, B., and Van den Bulte, C. (2011) 'Referral programs and customer value', *Journal of Marketing*, 75(1), pp. 46-59.

Sharp, H., Preece, J., and Rogers, Y. (2015) *Interaction Design: Beyond Human-Computer Interaction*. 4th edn. Chichester: John Wiley & Sons.

Smith, J. and Zhang, Y. (2022) 'Advances in optical character recognition for financial documents', *Pattern Recognition Letters*, 156, pp. 78-85.

Sommerville, I. (2016) *Software Engineering*. 10th edn. Boston: Pearson.

Stack Overflow (2024) *Developer Survey 2024*. Available at: https://survey.stackoverflow.co/2024/ (Accessed: 21 January 2025).

Stripe (2024) *Stripe API Documentation*. Available at: https://stripe.com/docs/api (Accessed: 22 January 2025).

Stripe (2024) *SEPA Direct Debit*. Available at: https://docs.stripe.com/payments/sepa-debit (Accessed: 21 January 2026).

Stripe (2024) *Strong Customer Authentication (SCA)*. Available at: https://docs.stripe.com/strong-customer-authentication (Accessed: 21 January 2026).

Thompson, R. (2021) 'The social psychology of money in relationships', *Journal of Social and Personal Relationships*, 38(7), pp. 2156-2178.

Verma, R. and Das, S. (2020) 'Efficient algorithms for minimal transaction settlement in group expense sharing', *Algorithmica*, 82(9), pp. 2567-2591.

Voigt, P. and Von dem Bussche, A. (2017) *The EU General Data Protection Regulation (GDPR): A Practical Guide*. Cham: Springer.

Wachter, S. and Mittelstadt, B. (2019) 'A right to reasonable inferences: Re-thinking data protection law in the age of big data and AI', *Columbia Business Law Review*, 2019(2), pp. 494-620.

Ware, C. (2012) *Information Visualization: Perception for Design*. 3rd edn. Waltham, MA: Morgan Kaufmann.

Weinman, J. (2012) *Cloudonomics: The Business Value of Cloud Computing*. Hoboken, NJ: John Wiley & Sons.

Xiao, J. J., Tang, C., Serido, J., and Shim, S. (2007) 'Antecedents and consequences of risky credit behavior among college students: Application and extension of the theory of planned behavior', *Journal of Public Policy & Marketing*, 30(2), pp. 239-258.

Zachariadis, M. and Ozcan, P. (2017) 'The API economy and digital transformation in financial services: The case of open banking', *Swift Institute Working Paper*, 2016-001.

Zelizer, V. A. (2017) *The Social Meaning of Money: Pin Money, Paychecks, Poor Relief, and Other Currencies*. Princeton, NJ: Princeton University Press.

Appendix

Fig 1. SplitSync (created with LM Arena AI)

Fig. 2 context diagram for Split-Sync (created with plantUML)

Fig. 3 Use case diagram for Split-sync (created with plantUML)

Fig. 4 Split-sync features (created with Claude AI)

Fig. 5 The ER diagram (created with plantUML)

Glossary

API (Application Programming Interface): Structured methods through which software components communicate, enabling Split-sync to integrate with external services like payment gateways and OCR engines.

Append-Only Ledger: Database architecture where records are never modified after creation, with changes represented as new transactions, ensuring immutable audit trails.

Background Sync: Browser API enabling queued operations to complete even when users close applications, essential for reliable offline functionality.

Balances View: Database view aggregating expenses and settlements to compute net amounts owed between group members.

Firebase: Google's mobile and web application development platform providing real-time database, authentication, cloud functions, and storage services.

Idempotency: Property where performing an operation multiple times produces the same result as performing it once, critical for reliable webhook processing.

IndexedDB: Browser database API enabling client-side data storage for offline functionality and caching.

Minimum Cash-Flow Algorithm: Graph optimization algorithm that minimizes the number of transactions required to settle all debts within a group.

OCR (Optical Character Recognition): Technology extracting text from images, enabling automated receipt data entry.

Open Banking: Regulatory framework and technical standards enabling third-party applications to access bank account data and initiate payments through standardized APIs.

PCI DSS: Payment Card Industry Data Security Standards defining requirements for systems handling credit card information.

Progressive Web App (PWA): Web applications using modern APIs to provide native app-like experiences including offline functionality, push notifications, and home screen installation.

Real-time Database: Database system that automatically synchronizes data changes to connected clients immediately without polling.

Service Worker: JavaScript code running in background separate from web pages, enabling offline functionality and push notifications.

Split: Database record allocating a portion of an expense to a specific group member.

Strong Customer Authentication (SCA): European payment regulation requiring multi-factor authentication for electronic payments exceeding risk thresholds.

Webhook: HTTP callback mechanism where external services POST notifications to application endpoints when events occur.

WebSocket: Communication protocol providing full-duplex channels over single TCP connections, enabling real-time data push from servers to clients.